

Speaker Notes

Contents

- [1 1. Finishing I/O, and advanced I/O](#)
- [2 2. Where we stopped](#)
- [3 3. Roadmap](#)
- [4 4. Takeaways](#)
- [5 5. Recap by prediction](#)
- [6 6. Predict: draw last week's stack](#)
- [7 7. It is not one buffer — it is a stack](#)
- [8 8. Explain: what dies with the process, what dies with the machine](#)
- [9 9. Predict now, and keep it for an hour](#)
- [10 10. What dies with the process](#)
- [11 11. Where Lecture 2 stopped](#)
- [12 12. Three lines that print nothing](#)
- [13 13. Why the file is empty](#)
- [14 14. What this demonstration does not prove](#)
- [15 15. Predict: what will `ltrace` show that `strace` did not?](#)
- [16 16. `ltrace` : the same idea, one layer up](#)
- [17 17. Which levels can you actually watch?](#)
- [18 18. The same write, at three levels](#)
- [19 19. The same write, at three levels](#)
- [20 20. Meet the program: `iodemo.c`](#)
- [21 21. Three calls in the source. One in the trace.](#)
- [22 22. `fwrite` takes a size and a count](#)
- [23 23. Two operating systems, side by side](#)
- [24 24. Process Monitor: what the kernel did](#)
- [25 25. NtTrace: a tracer for the Windows Native API](#)
- [26 26. Three Windows words you will need](#)
- [27 27. Predict: the same program, on Windows](#)
- [28 28. A real Windows trace](#)
- [29 29. A handle is not a file descriptor](#)
- [30 30. Two error namespaces in one return value](#)
- [31 31. The stable boundary sits at a different level](#)
- [32 32. Where the boundary sits is contested history](#)
- [33 33. Linux is the outlier, not Windows](#)

34 34. One program, two kernels

35 35. What we have not looked at

36 36. `write()` returned success. The bytes are not on a disk.

37 37. Buffering moves your errors

38 38. Predict: all your writes succeeded — can the close still fail?

39 39. Why a close can fail when every write succeeded

40 40. Observe: close it twice, and watch what the second close hits

41 41. Never retry a failed close

42 42. Networking and IPC are this same mechanism

43 43. Honest limits

44 44. Honest limits, continued

45 45. Lecture 2 ends here. The question does not.

46 46. So when is it safe?

47 47. The kernel's own count of what it has not written

48 48. Observe: the program finished at zero seconds

49 49. Five runs, three sizes

50 50. Five seconds in memory that belongs to nobody

51 51. Which rule fired, and how we know

52 52. Visibility is not durability

53 53. Predict: what does one `fsync` per record cost?

54 54. Observe: sixty times, for the same hundred writes

55 55. What `fsync()` promises, and what it does not

56 56. Windows: the same shape, and now measured

57 57. The two kernels write in opposite orders

58 58. Predict: your `fsync()` failed — what does the retry return?

59 59. Observe: the retry returns success, over data that was never written

60 60. Explain: the retry does not retry

61 61. What the engines did instead

62 62. Windows here is not "the same", and not "safe"

63 63. Sync the directory too

64 64. Write-then-rename, and why it needs a sync

65 65. The respectable opposite view

66 66. Log first — and what if the log write fails?

67 67. So when is it safe?

68 68. Lecture 4 starts here

69 69. What a pipe is

70 70. Blocking, non-blocking, and letting the kernel wait for you

71 71. Predict: a hundred pipes, and nobody writing

72 72. Where the question comes from

73 73. A thread each, and why that stopped working

74 74. Observe: two seconds, zero bytes, counted

75 75. What those counts do and do not say

76 76. `epoll` is three calls

77 77. Level-triggered and edge-triggered

78 78. Three ways to wait, and we watch all three

79 79. Readiness is permission to try, not a promise

80 80. The real difference is who owns the buffer

81 81. It is not a Linux/Windows split

82 82. The trap this demonstration sets

83 83. Four Windows words before the Windows pair

84 84. Predict: the same hundred pipes, on Windows

85 85. Observe: Windows, traced, both ways

86 86. Which numbers you may compare, and which you may not

87 87. What a ring is

88 88. Where both platforms ended up

89 89. Same mechanism, different clothes

90 90. Predict: which of the four calls changes?

91 91. Observe: the file run and the pipe run

92 92. The descriptor is the abstraction, not the file

93 93. Predict: the same program, writing to your terminal

94 94. Observe: three writes, not one

95 95. Explain: the destination changed the trace, at the C library, not the kernel

96 96. Predict: the network leg

97 97. Observe: both netcats, traced

98 98. Then what are `send` and `recv` for?

99 99. The same interface is not the same contract

100 100. Predict: the same three destinations, on Windows

101 101. Observe: the API changes the call, the object does not

102 102. Windows: a `SOCKET` is not a `HANDLE`

103 103. Bytes you never touch

104 104. Predict: three buffers, one message

105 105. Observe: three lines become one

106 106. Gather buys crossings, not copies
107 107. Two promises, and only one of them is unconditional
108 108. Predict: moving a file you never look at
109 109. Observe: half the calls, and no trip through your program
110 110. What the name does not cover
111 111. Zero-copy moves bytes faster. It does not make them safer.
112 112. Windows, and a silent fallback
113 113. Homework, and the next question
114 114. Assignment 1 is not set today
115 115. Voluntary homework, and three rules for it
116 116. Summary — what follows from this
117 117. What Lecture 4 answers
118 118. References

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Slides: 118 · **Duration:** 90 min · **Venue:** Faculty of Informatics, Masaryk University (FI MUNI)

11. Finishing I/O, and advanced I/O

On screen:

PB158 · Lecture 3

Your program never talks to the disk, the network, or another program. It talks to the layer next to it. Every layer says done about something different [V50] [inference].

*Ondřej (Ondra) Krajíček me@ondrejkrasicek.com · www.linkedin.com/in/OndrejKrajicek
Built at render time*

Lecture 2 followed one write call down the stack. It stopped at slide thirty-two, on an open question.

Today has two halves. The first half finishes Lecture 2. Nothing is dropped and nothing is rewritten. The second half answers the question Lecture 2 left open.

The sentence on the screen is the whole lecture. A program never talks to the disk. It talks to the layer next to it. That layer returns, and the return value says done. That word names a different place at every layer.

The cover sentence carries a hedge. The source says almost every fast interface behaves this way, not every one [V50]. The sharper form on the cover is the speaker's own [inference].

One question runs through the whole hour. Whose done was measured? A layer returns when it has accepted responsibility for the work, not when the work has happened [V50]. The question returns three times today, and it stays useful after this course.

Every demonstration today is a live run on this machine. There are no screenshots. A failure is visible to the room.

2 2. Where we stopped

On screen:

*PB158 · Lecture 2**Lecture 2 left four claims. Today asks each of them whose done that was [V50].*

- 1. **Buffering pays for adjacent writes** [P1] — a seek forces the buffer out [inference].*
- 2. **A library call is not a system call** [P2] — `printf` shows up in `ltrace`, `write` in `strace` [V10].*
- 3. **The boundary is observable** [P3] — a tracer prints the call and its return value [S1].*
- 4. **Two kernels, two boundaries** [P4] — "the disagreement is about which layer is nominated" [V8].*

*Today adds one question to each: whose done was that? [V50]**[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>*

Lecture 2 stopped at slide thirty-two. These four claims are what the room carries out of it [P1] [P2] [P3] [P4].

Claim one is the one Assignment 1 measures [inference]. Claim two is why `printf` appears in one tracer and `write` in another [V10]. Claim three is the method of this course, and every number today comes from it [S1]. Claim four returns twice in the tail, at the waiting models and at the socket [V8].

The new question is on the last line. A layer returns when it has taken responsibility, not when the work has happened [V50]. Each of these four claims meets that question today.

3 3. Roadmap

On screen:

*PB158 · Lecture 3**Two blocks. The first finishes Lecture 2. The second is announced, and may run into Lecture 4.*

- **Parts 1 to 5 · planned** — Lecture 2, finished from the slide it stopped on. Nothing is dropped.*
- **Part 6 · planned** — so when is it safe? The question Lecture 2 left open.*

• **Parts 7 to 9 • the announced tail** — waiting for input, three destinations, bytes you never touch.

• **Part 10 • planned** — voluntary homework, and the question Lecture 4 answers.

The tail is built in full. If the clock stops us, Lecture 4 opens on it.

The hour has two blocks. Parts 1 to 5 finish Lecture 2 from its slide thirty-three. Part 6 answers the question that lecture ended on.

Parts 7 to 9 are the announced tail. The tail is built in full and it is not filler. Lecture 4 carries it if this room does not reach it.

Part 10 sets the voluntary homework and names the next question. Part 10 runs whatever the clock has done to the tail.

4 4. Takeaways

On screen:

PB158 • Lecture 3

1

Acknowledgement is not durability [T1]

A call that reports success means the layer below accepted the bytes. The device keeps its own cache.

2

Readiness and completion [T2]

Two different answers to one question: how does a program wait for I/O? Tail.

3

A file, a pipe and a socket [T3]

One write carried a file, a pipe and a socket. Tail.

4

One call, many buffers [T4]

The kernel can move bytes the program never touches. Tail.

These are the four claims to carry out of the room. Each one is marked again on the slide that earns it.

Takeaway one is in the planned block, and every student needs it [T1]. Takeaways two, three and four sit in the announced tail [T2] [T3] [T4].

All four have the same shape. A call returns, the return value says done, and the word names a different place each time.

5 5. Recap by prediction

On screen:

Part 1

Three minutes, on paper, before I show you anything.

Three minutes. Paper out, laptops closed.

This part does not put last week's diagram back on the screen. A diagram read out to a room produces agreement now and no recall in an hour.

The room draws the diagram first. Lecture 2's own version follows two slides later. The gap between the two is what each student kept.

6 6. Predict: draw last week's stack

On screen:

Part 1 · Recap

Three minutes. Nothing on the screen until you have written something down.

1

Draw it

The four levels between your program and the medium. Name each one.

2

Mark it

After fflush() returns, which level are the bytes in?

3

Answer it

The machine loses power right now. What is lost?

Write all three. Nothing is handed in. I will ask several of you to read yours out [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Three tasks. Three minutes.

First, draw the stack. Four levels between the program and the medium, each one named. Three levels are enough if the fourth does not come back. Leave a gap for it.

Second, mark where the bytes are after `fflush()` returns. One level. Circle it.

Third, and this is the task that matters today. The power goes out this second. What is lost?

Nothing is handed in. Ask two or three students to read their answers out. The wrong answers are as useful here as the right ones. A written wrong prediction is worth more than a copied right answer [S51].

Three minutes on the clock. Watch the clock, not the paper.

Then: pens down, look up.

The next slide carries Lecture 2's own diagram, unchanged. Each student puts it beside the drawing on the page.

7 7. It is not one buffer — it is a stack

On screen:

Part 1 · Recap

[code]

Levels 1 and 2 are alternative routes into level 3 [V6]. Each arrow is one call [S17] [S3] [S13] [V37] [V33].

[S17] `io.IOBase.flush()` — Python documentation. [S3] `fflush(3)` — Linux man-pages.

[S13] `fsync` — The Open Group.

Four levels. Read them from the top [inference].

Level one is the program's own buffer, owned by the language runtime. Level two is the C library buffer. Level three is the operating system memory for file data. Level four is the cache inside the storage device. The medium is at the bottom.

Levels one and two sit side by side. They are alternative routes, not a pile. A Python program uses the buffer at level one and calls the kernel directly. A C program uses the buffer at level two. This detail returns in four slides, because it is the one people get wrong [V6].

Now read the arrows. Each arrow is a different call. Each call moves the bytes down exactly one step. No single call takes data from the top to the bottom.

The bottom arrow also returns later. Part 6 does not skip it.

8 8. Explain: what dies with the process, what dies with the machine

On screen:

Part 1 · Recap

Two different failures. Part 2's demonstration tests one. Your recap question asked about the other.

The process dies

Levels 1 and 2 go with it. Anything already handed to the kernel survives, because the kernel writes it back on its own [V14].

The machine loses power

Levels 1, 2 and 3 are gone. Level 4 goes too, unless the device has power-loss protection [inference].

Part 2 opens with `os._exit(0)`. That is the process dying, not the machine [inference].

Part 6 measures the level-3 window.

This slide answers the third recap question. It is also the bridge into the rest of the hour.

Two failures, and they are not the same failure.

Kill the process. Levels one and two are that process's memory, so they die with it. Level three is the kernel's memory, and the kernel still runs. The kernel writes those bytes back [V14]. The process does not need to exist.

Pull the power. Level three dies too, because it is RAM. Level four is the cache inside the drive. Survival there depends on power-loss protection, which is capacitors inside the drive. Most consumer drives do not have it, and that is a reading rather than a figure with a source.

The distinction arrives before Part 2 for one reason. Part 2 opens with an exit call that skips cleanup. The file comes out empty. That demonstration loses the level-one buffer only. It says nothing about levels three and four [inference]. Slide fourteen states that bound.

No machine loses power in this room today. Part 6 measures how long level three holds the bytes instead. That interval is the window a power cut would catch.

9 9. Predict now, and keep it for an hour

On screen:

Part 1 · Recap

The prediction for Part 6. You write it now, before Part 2, so you carry the question through the carried hour.

[code]

The question

The process has exited. How long until the kernel actually writes your bytes to the device?

Your options

Under a second · about five seconds · about thirty seconds · not until shutdown.

One of four. Write it in the corner of your page. Do not change it later [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

One more prediction, and then Lecture 2 resumes.

The prediction is collected now rather than in forty minutes. The reason is recall across a long lecture. The start and the end of an hour stay, and the middle blurs. Part 6 is the middle. An open question carried through Parts 2 to 5 arrives at Part 6 already owned by the room, instead of arriving cold.

The program on the screen is the one every student has written. It opens a file, writes a hundred records with the default buffering, prints done, and exits. It never flushes. It never calls `fsync()`. That idiom "contains no durability step anywhere" [V4].

The full file is in the demo folder. The four lines here carry the whole question.

The process is gone. The kernel still holds the bytes. How long does it keep them?

Four options. Under a second. About five seconds. About thirty seconds. Not until shutdown.

Each student writes one option in the corner of the page. Collect the answers out loud before any run. Then hold the room to them.

The demo folder holds the answer. Nobody opens it before then.

10 10. What dies with the process

On screen:

Part 2 · What dies with the process

Parts 2 to 5 are Lecture 2, finished. Same slides, same order, renumbered into today.

Part 2. This is Lecture 2, finished, and it runs to Part 5.

The slides are last week's, carried across without a word changed. Same order, same sources. Only the part labels are today's, so the room tracks one set of numbers rather than two.

Seven slides in this stretch are new. Two are predictions Lecture 2 never collected. Three finish the close story, which Lecture 2 asserted and never measured. Two are the bridges in and out.

One Lecture 2 slide is absent. Its title was what to do on Monday. That is a conference-talk ending, and a course does not need it. Its content was the assignment hand-off, and Part 10 does that properly.

Lecture 2 stopped at slide thirty-two, the buffer stack now on paper. The next slide states exactly that. Slide thirty-three then starts with a program that prints nothing.

This is the longest part of today by a wide margin. Finish it rather than summarise it.

11 11. Where Lecture 2 stopped

On screen:

Part 2

Last Tuesday ended after slide 32: the device's acknowledgement comes early [V20], and no tool in the room can check that report [V5].

From here to the end of Part 5 the slides are Lecture 2's, as built. The clock ran out after slide 32. Slide 33, the `os._exit` demonstration, is where we pick up [inference].

Your two predictions from Part 1 stay on your page: the level the bytes are in after `fflush()`, and what a power cut takes. The first demonstration answers a different question — what a process dying takes — so keep both apart [inference].

Lecture 2 ended after slide 32. That slide said the device acknowledges a write early [V20]. No tool available in this room separates an honest write to the medium from a fast lie [V5]. Today picks up there.

Everything from here to the end of Part 5 is Lecture 2 as built. The text is unchanged, and the slides inside it keep their numbers. Only the part labels at the top are new, so this lecture reads as one sequence.

Two answers are already on paper from a few minutes ago. Keep them. The first demonstration is about a killed process, which is a different failure from a machine losing power. Hold that distinction. Part 6 returns to it.

12 12. Three lines that print nothing

On screen:

Part 2

A demonstration. You have the stack now, so predict this one.

[code]

To a terminal

Does hello appear?

Redirected to a file

What does the file contain?

Write down both answers before I run either one [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Three lines. Import, print, exit [inference]. Nothing unusual anywhere.

Same method as Part 1: predict first, out loud. Most of the room gets this one right, which is why it sits here rather than at the beginning.

Two runs. First in a terminal. Then redirected into a file, and the file goes on the screen.

Thirty seconds. Two answers. Does hello appear on the terminal, and what is in the file?

Collect both by a show of hands: the terminal case first, then the file.

Run the terminal case. Then run the redirected case.

The file is empty. Zero bytes. A correct prediction here is not a guess. It comes from the diagram at the start of this part, and that diagram is what the demonstration tests.

13 13. Why the file is empty

On screen:

Part 2

The rule sits in the C standard, not in the kernel.

1

Mode follows destination

Output is "fully buffered if and only if the stream can be determined not to refer to an interactive device" [S5].

2

A terminal is interactive

Bytes go out per line [inference].

3

A file is not

Bytes wait [inference].

4

The exit skips cleanup

No write reaches the kernel [inference].

[S5] C99 draft §7.19.3. <https://busybox.net/~landley/c99-draft.html>

Here is the explanation, and notice where it comes from [S5].

The rule is in the C language standard. It is not an operating system behaviour and it is not a Python quirk. The standard says that standard output is fully buffered if and only if the stream can be determined not to refer to an interactive device.

Then the third line ends the process without running any cleanup. The standard explicitly permits termination paths that do not close files properly. The buffer is process memory. The process is gone. The bytes went with it.

Now the sentence that connects this back to section two. A trace of this program holds no write call at all. Not a failed one. Not a short one. None. The bytes never got as far as the operating system.

That is the top of the stack, proved to exist by removing it.

14 14. What this demonstration does not prove

On screen:

Part 2

It proves level one is real. It proves nothing about levels three and four.

"the honest classroom demo is a power-cut, not a `kill -9`. Killing the process loses only the runtime's buffer; the kernel still writes back." [V14]

1

What died

The process, and the level-one buffer with it [inference].

2

What survived

Anything already in the page cache [V14].

3

Wrong conclusion

"So flush() was enough." It was enough against this test [inference].

The demonstration needs a bound at once, because it is easy to over-read.

A kill destroys that process's memory, and that is level one. Bytes already handed to the operating system sit in the page cache, and the operating system writes them back in its own time [V14]. The process does not need to exist.

So the demonstration proves that level one exists and can lose data. It proves nothing about levels three and four.

The honest test for the lower levels is a power cut, not a kill [V14]. A power cut shows what the page cache was holding. Nobody runs that test in this room today.

The care has a reason. A student tests with a kill, watches the file survive, and concludes that the program flushed everything. That conclusion holds against a kill and fails against a power failure. Same demonstration, and the difference is which level the conclusion claims to have tested.

15 15. Predict: what will `ltrace` show that `strace` did not?

On screen:

Part 2

Before I run the second tracer. Two lines on your page [S14].

*It records the **library calls** a program makes [S31] — here, `fwrite` in the C library [inference]. `strace` sees `write` in the kernel instead [inference].*

1

The C program calls `fwrite` three times

How many `fwrite` lines does `ltrace` print?

2

The kernel saw one write

Does `ltrace -S` print that line too, or only the library calls?

The answer follows from the stack you drew twenty minutes ago [inference].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S31] `ltrace(1)`. <https://man7.org/linux/man-pages/man1/ltrace.1.html>

Two numbers, collected before the second tracer appears.

One word first, so the question means something. The second tracer is `ltrace`. It records the calls a program makes into shared libraries [S31]. That means `fwrite` in the C library, not `write` in the kernel [inference].

The program calls `fwrite` three times. How many `fwrite` lines does the library tracer print? Each student writes a number.

Second question. The kernel already showed one `write`. The `-s` option adds system calls to the output. Does that `write` line appear?

This is not a trick. Both answers follow from the diagram drawn at the start of the hour [inference]. The library tracer watches the boundary above the library. The system-call tracer watches the boundary below it. Three calls go in at the top. One call comes out at the bottom.

A student who answers both already holds the takeaway of Part 2, before the tool reaches a slide.

Thirty seconds. Both numbers on paper.

16 16. `ltrace`: the same idea, one layer up

On screen:

Part 2

strace stops at system calls. ltrace stops at calls into shared libraries [inference].

What it does

It "intercepts and records the dynamic library calls" a process makes [S31]. It can also print "the system calls executed by the program" [S31].

Where it comes from

Juan Cespedes wrote it originally [S32]. The manual page names Petr Machata too [S31]. On Debian and Ubuntu you run `apt-get install ltrace` [S32].

How to use it

[code]

It runs on Linux only [inference].

[S31] `ltrace(1)`. <https://man7.org/linux/man-pages/man1/ltrace.1.html> [S32] `ltrace`. <https://ltrace.org/>

Here is the second tracer. It makes the next slide checkable.

What it does. Its manual page says it intercepts and records the dynamic library calls the process makes, and that it can also print the system calls underneath [S31]. The page says its use is very similar to `strace` [S31]. So this is the idea you already have, one layer up [inference].

Where it comes from. The project page says Juan Céspedes wrote it originally [S32], and the manual page names Petr Machata as well [S31]. The same page gives the Debian and Ubuntu command [S32].

How you use it. With `-s` you get the system calls as well as the library calls, which is the option that matters here [S31]. Try it yourself. Run both tracers on one program. Then watch the two outputs disagree.

17 17. Which levels can you actually watch?

On screen:

Part 2

I promised three of four boundaries. Here is the tool each needs.

BOUNDARY	WHAT WATCHES IT	IN THIS COURSE
<i>App → library</i>	<code>ltrace</code> sees it, <code>strace</code> does not [V10]	<i>the tool above</i>
<i>Library → kernel</i>	<code>strace</code> — where our tracer attaches [S1]	<i>today</i>
<i>Kernel → device</i>	<i>a tracer for the kernel's block layer</i> [V30]	<i>elsewhere</i>
<i>Device → medium</i>	<i>no system-call trace shows whether the device flushed</i> [V5]	<i>no tracer</i>

The first slide claimed three of the four boundaries are observable. This slide names which three, and why the fourth is not one of them [V5].

The diagram has four boundaries. Each row gives the tool that boundary costs.

The top boundary needs the tool from the previous slide. The source states the pair directly: `printf` is visible to `ltrace` and `write` to `strace` [V10]. The demonstration program calls `fwrite` instead. Running both tools tests takeaway two rather than asserting it.

The second boundary is the one under observation all hour.

The fourth boundary is not observable from software at all.

So three of the four are watchable, one of them with a tool now in hand. The unwatchable one is where data actually gets lost. Section six returns to that asymmetry.

18 18. The same write, at three levels

On screen:

Part 3 · One write, three levels

Now that the levels have a reason to exist, walk down them with one small program.

Eight minutes. Section three established that the levels exist, and why. This section walks down them with a program every student can compile.

C is part of this programme, so the code needs no apology and no detour through pointers. The core of the program goes on the screen. It compiles live, runs under the tracer, and the room reads the trace.

19 19. The same write, at three levels

On screen:

Part 3

LEVEL	PYTHON	C	BUFFERED?
Application	<code>f.write(b"x")</code> [S6]	—	yes [S6]
C library	—	<code>fwrite(...)</code> [S3]	yes [S3]
System call	<code>os.write(...)</code> [S20]	<code>write(...)</code> [S2]	no [inference]

The bottom row takes a **file descriptor** — the small integer naming an open file. You saw one already: the `= 3` in the trace line was a file descriptor [S1].

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S2] `write(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S3] `fflush(3)` — Linux man-pages. [S6] Python documentation, `open()`. [S20] `os` — Miscellaneous operating system interfaces, `os.write()`. <https://docs.python.org/3/library/os.html#os.write>

Read the table by column first, then by row.

The Python column and the C column are two routes to the same bottom row. A Python file object holds a buffer at level one. `fwrite` fills a buffer at level two. Both reach `write` in the end, and `write` is the system call.

Now read the last column, because it carries the point. Each level holds a buffer, or it passes the bytes straight down. A Python file object holds one. The `os.write` function does not, because it wraps the system call directly [S20]. The levels are not decoration. They are the mechanism Part 1 measured.

The bottom row introduces a term this deck has deferred until now. A file descriptor is the small integer a Linux program uses to name an open file in system calls. One has appeared already: the `open` call returned three [S1]. It is the first argument of every `write` to that file.

Keep the word. In section five it is the first thing that differs.

20 20. Meet the program: `iodemo.c`

On screen:

Part 3

Nine lines of C. The first open fails on purpose, so the trace shows an error path.

[code]

It prints nothing, on purpose. A program that prints nothing adds no `write` lines of its own to the trace [inference].

Nine lines on the screen. The full file adds comments and error returns.

Step one opens a file that does not exist, and that failure is deliberate. If every call succeeds, the trace is success all the way down, and the most interesting difference between the two platforms never appears. Section five shows that difference.

Step two opens the file the program means to write.

Step three writes fourteen bytes in three calls. The C library collects them in its own user-space buffer before any of them crosses into the kernel [S3] [S5]. Watch that number.

Step four closes the file and checks the return value of the close. Almost no published C does that. Section six gives the reason.

The program prints nothing at all. The reason is the plain one. Every `write` line in the trace then comes from the three `fwrite` calls, and not from a print statement [inference]. A silent program also makes a trace that is easier to read.

Compile it now, with warnings on. Then run it.

21 21. Three calls in the source. One in the trace.

On screen:

Part 3

3

fwrite calls in the source

14

bytes written in total

1

write call in the trace

The three `fwrite` calls go into one C library buffer. The manual calls that level "the user-space buffers provided by the C library" [S3]. The line to look for in the live trace, predicted from the source [inference]:

[code]

The 3 is the file descriptor. The 14 is all three `fwrite` calls, in one crossing [inference].

[S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

This is where takeaway two stops being a claim and becomes something on a screen.

Three `fwrite` calls in the source. Fourteen bytes. One `write` in the trace.

Point at the source, then at the trace. There is no line-by-line correspondence between them, and there never was. The source holds a library call. The operating system sees a system call. The source never states how many system calls to make. The buffering policy in the library fixes that number [inference].

The line on the slide is a prediction, written from the source before any run. The real trace comes next. Check it against the prediction. A difference means the prediction was wrong, not the machine.

Notice the moment of the write as well. It is not the third `fwrite`. It happens inside `fclose`, because `fclose` hands the buffer down to the kernel. The call in the source and the line in the trace are not even at the same point in time.

22 22. `fwrite` takes a size and a count

On screen:

Part 3

Four arguments, and two of them multiply.

[code]

1

It returns a count

Items written, not bytes. The count equals bytes only when the item size is 1 [S21].

2

Fewer is legal

Below it, "a successful write() may transfer fewer than count bytes" [S2].

[S2] write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S21] fread(3), which documents fwrite(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>

A small detail that trips people. It costs thirty seconds now, or twenty minutes at the keyboard later.

The call takes four arguments: a pointer to the data, the size of one item in bytes, the number of items, and the stream. The demonstration program passes one and six, which means six items of one byte each. Six and one would write the same bytes.

The order matters for two reasons.

First, the return value counts items, not bytes. The manual says the count equals the number of bytes only when the item size is one [S21]. Six items of one byte can return four, and four bytes went out. One item of six bytes can return zero, and nothing went out.

Second, fewer is legal. The system call underneath may transfer fewer bytes than requested [S2], and the library call may report fewer items than requested [S21]. Two different contracts, one habit. Read the return value.

23 23. Two operating systems, side by side

On screen:

Part 4 · Two kernels

Read as a comparison. Try the tools yourself afterwards.

Twelve minutes for the section many students came for.

A warning about what this section is not. It is not a demonstration that one platform is better. It is not a tutorial on installing a Windows tracer. It is a reading exercise on two pieces of output. The lesson is a structural difference between the systems, not a difference in formatting.

The evidence also needs care. Each capture on the following slides carries its origin and its limits on the slide itself.

24 24. Process Monitor: what the kernel did

On screen:

Part 4

It reports filesystem and registry activity [S18], which is a different cut than the system-call boundary [inference].

What it does

It shows "real-time file system, Registry and process/thread activity" [S18]. That view shows filesystem and registry activity, not system calls [inference].

Where it comes from

Mark Russinovich wrote the page, in Microsoft's Sysinternals documentation [S18]. The tool "combines the features of two legacy Sysinternals utilities, Filemon and Regmon" [S18]. You download it. You can also run it from Sysinternals Live [S18].

How to use it

[code]

[S18] Process Monitor. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

Two Windows tools, one after the other.

This is the first. Microsoft describes it as showing real-time file system, Registry and process activity [S18]. That view shows filesystem and registry activity, not the system calls a program made [inference].

Where it comes from. Mark Russinovich wrote the page, in Microsoft's Sysinternals documentation [S18]. Microsoft says the tool combines two older utilities, Filemon and Regmon [S18]. Download it, or run it from Sysinternals Live without an install [S18]. The same page also links a Procmon for Linux [S18].

How to use it. It captures everything at once, so filter by process name first. Then read the Operation column. The demonstration program produces `CreateFile` and `WriteFile` rows [inference]. Check that on your own machine.

25 25. NtTrace: a tracer for the Windows Native API

On screen:

Part 4

It watches the native API, "the interface between the application space and the OS kernel" [S10].

What it does

A "simple trace facility for the Windows Native API", "roughly equivalent to strace on Linux" [S10]. It attaches as a debugger, so it "needs no driver and no administrator rights" [V21].

How to use it

[code]

Try both yourself on `iodemo.exe` : Process Monitor shows what the filesystem did, NtTrace shows what the program asked for [inference].

[S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/> [S33] NtTrace source. <https://github.com/rogerorr/NtTrace>

The second tool is the one this lecture reads.

What it does. Its project page says it provides a simple trace facility for the Windows Native API, and that it is roughly equivalent to `strace` on Linux [S10]. That comparison belongs to the tool's author, and the room should know whose sentence it is. The same page says the tool uses the Windows debug interface to intercept the returns from the native API [S10]. The consequence, from the reading note: it needs no driver and no administrator rights [V21]. That one choice is why it runs on a student laptop at all.

Where it comes from. Roger Orr wrote it and still maintains it. The project page lists releases from 2011 to February 2026 [S10]. The source is on GitHub under the MIT licence [S33].

How to use it. The page's own example filters on the substring `File`, which is the form this lecture wants [S10]. One honest word: this tool is one person's work, and Microsoft calls those entry points "subject to change from one release of Windows to another" [S9].

Run both tools on the same program. That is the fastest way to see that they watch different layers. Process Monitor reports filesystem events. NtTrace reports the native calls the program made [inference].

The claim needs precision, because it is easy to overstate. NtTrace does not attach at the same point as the Linux tracer. The Linux tracer sits at a documented kernel boundary. NtTrace sits one level above the kernel, in a user-mode library. That gap is the lesson of this section [S10].

Neither of today's tools is what people run in production, and the closing part returns to that.

26 26. Three Windows words you will need

On screen:

Part 4

[code]

1

Win32

The documented interface a Windows program calls. The 32 is history, not a width [inference] [S22].

2

The native API

The layer below Win32, inside ntdll.dll. Microsoft calls it "internal to the operating system" [S9].

3

The object manager

The kernel component that names open objects. Its paths begin ??\ [V22].

[S9] winternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation> [S22] Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>

Three words, thirty seconds, and then the Windows trace makes sense. The picture is the same stack you drew in Part 1, seen from the side: your program, two libraries, then the boundary into the kernel.

Win32 is the documented interface a Windows program calls. It is the Windows equivalent of the C library plus the POSIX calls. It is the interface the documentation tells you to use. The name is historical.

Microsoft renamed the same thing the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22]. So the 32 in the older name tells you nothing about the width of a program or of a trace [inference]. The trace you see next came from a sixty-four bit process [S36], and the name Win32 still applies to it [inference].

The native API is the layer below Win32. It lives in a library called ntdll, and it is the last user-mode code before the kernel. Microsoft's own documentation calls it internal to the operating system [S9]. NtTrace attaches at this layer, which is why its lines start with Nt [S10].

The object manager is the kernel component that names open things — files, processes, events. Its paths look different from the paths you type. You see one later in this section, in a trace of the Windows command shell [S34].

27 27. Predict: the same program, on Windows

On screen:

Part 4

Before the Windows capture goes on the screen. Two answers, on paper [S14].

1

The three fwrite calls

How many write calls does the Windows trace show — three, or one?

2

The deliberate failure

Linux reported -1 ENOENT. How many error codes does Windows report for the same failed open?

The first answer tests whether buffering is a C library property or a platform property. The second one has a surprise in it.

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Last prediction. Write it down.

Same source file. Same three `fwrite` calls. Compiled with the Microsoft compiler, and the trace comes from Windows [S36].

First question. How many write calls does the Windows trace show? Three, or one?

The buffering on Linux happened in the C library, and the C library is not the kernel. So the answer separates a Linux fact from a C fact.

Second question, and this one is harder. The first `fopen` fails on purpose. Linux answered minus one and the name `ENOENT`. One error, one namespace. How many error codes does Windows report for the same failure?

Most of the room answers one. Collect the written answers first. Then show the capture. Thirty seconds.

28 28. A real Windows trace

On screen:

Part 4

You can already read this. The anatomy matches the Linux line, and the vocabulary differs [inference].

[code]

iodemo.exe under NtTrace, 2026-09-15 — the same four steps as the Linux trace [S36] [S37]. Ignore every argument except the handle, the file name and the return value [inference].

[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/iodemo-c/README.md

This is the course's own program on Windows. The trace of `iodemo.exe` comes from `NtTrace64.exe` on 15 September 2026. The Linux trace of the same program comes from the same day [S36] [S37].

Read both as the four steps from part four [S36].

Name, arguments in parentheses, return value after the arrow. Same anatomy as the single Linux line the room already read.

One thing does differ. Here `=> 0` means success, and a return value starting `0xc` is a failure code [S36] [inference].

The three dots mark arguments cut from the slide [S36].

The capture runs to 45 lines [S36]. These are the last four calls in it. Every call above them belongs to the loader, which opens its own files before `main` runs [S36] [inference].

The first call wraps over two printed lines, so count calls and not lines. Call one is the deliberate failure, and its return value carries two error codes [S36]. Call two opens `iodemo.out` and returns handle `0x124` [S36]. Call three writes `Length=0xe` bytes, which is fourteen — three `fwrite` calls in one crossing [S36] [inference]. Call four closes the handle [S36].

One argument looks unfamiliar. `ObjectAttributes=0x58:"iodemo.out"` means the program named the file relative to an open directory [S36] [inference]. So neither of the program's two opens carries the object-manager prefix [S36]. The loader's calls above do carry it [S36]. The example on a later slide comes from the shell [S34].

29 29. A handle is not a file descriptor

On screen:

Part 4

First difference, visible in the first argument of almost every line.

Linux

A file descriptor. A small integer indexing a per-process table. Our trace shows = 3 [S37].

Windows

A handle. An opaque value naming an open object. Our trace shows `FileHandle=0x124` [S36].

Same job in a trace, different mechanism in the system. Do not use one word for the other [inference].

[S36] Our NtTrace64 capture of `iodemo.exe`, 2026-09-15 — `demo/iodemo-c/NtTrace64-IODEMO-Output.txt` [S37] The container capture of `iodemo`, 2026-09-15 — `demo/iodemo-c/README.md`

First difference, and it sits in the first argument of nearly every line on both systems [S36] [S37].

One program, one file, two systems. Linux answered `3`, and Windows answered `0x124` [S36] [S37].

Linux returns a file descriptor. It is a small integer indexing a table that belongs to the process. Three, four, five. The numbers support reasoning.

Windows returns a handle. It is an opaque value that names an open object, and the object need not be a file. It can be a process, an event or a registry key.

In a trace the two occupy the same position and do the same job, which makes one word for both tempting. This course keeps the two words apart. The word handle in a Linux assignment gets one question back: which system does this mean?

This is a small point. It gets a slide because it is the cheapest instance of today's platform lesson: the same job, two mechanisms, visible in one argument.

30 30. Two error namespaces in one return value

On screen:

Part 4

Second difference, and it is the layer boundary showing up where nobody expects it.

[code]

1

First number

An NTSTATUS, the native layer's code [S36] [inference].

2

Second number

The Win32 error it translates into [S36] [inference].

3

Linux

One errno: -1 ENOENT [S37].

[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/iodemo-c/README.md

Second difference, and it is the most interesting of the three.

That return value is the program's first line, the `fopen` designed to fail [S36].

Look at it. It carries two error codes at once. The first is the native layer's own code. The second is the Win32 error the first translates into, with the message text a user would actually see.

Two namespaces, in one return value, from one call.

Linux has one. The same failed open returns `-1` and the name `ENOENT` [S37]. That number is the error, and there is nothing to translate, because only one layer names errors.

Windows has two because two layers genuinely exist there, and both are real interfaces with their own error vocabularies. The line shows the seam between them. It is visible because the tracer sits below the translation and reports both sides.

This is why the demonstration program opens a file that does not exist. Without a failure this line never appears, and a clean run hides the seam between the two layers.

31 31. The stable boundary sits at a different level

On screen:

Part 4

Third difference, and the other two are symptoms of it.

In our trace of the Windows command shell a path reads `"\\??\\C:\\WINDOWS\\system32\\c_852.nls"` [S34]. That prefix "is the object-manager namespace, which a Win32 program never types" [V22].

Linux

[V8]

The system call interface is the public contract.

Syscall

Windows

[S9]

Native entry points are "subject to change from one release of Windows to another".

Win32

[S9] `winternl.h` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>

Third difference, and the other two are symptoms of this one [inference].

The path on this slide comes from the shell's trace [S34]. The demonstration program's two opens name their file relative to an open directory, so neither of them carries the prefix [S36].

The path starts with a backslash, two question marks, a backslash. No Win32 program ever types that string. It is the name of the file in the object manager's namespace, which sits below the interface a Windows programmer uses. The tracer shows a layer programs do not normally see.

Microsoft says so in its own documentation. The native entry points are internal to the operating system and subject to change between releases [S9]. That is a first-party statement.

On Linux the opposite holds. The system call interface is the public contract. A program may write a system call number down and rely on it.

Now the careful sentence, and it is the takeaway. The word nominate means one thing here: the vendor documents that layer as the one it will not break. Both systems are stable at the layer they nominate, and the disagreement is about which layer that is. Everything else follows from that single placement, including where a tracer can attach.

32 32. Where the boundary sits is contested history

On screen:

Part 4

Two readings survive the evidence. I will not pick one for you.

1

Reading A: deliberate

POSIX, OS/2 and Win32 each reached NT through the native API [S19].

2

Reading B: vestige

A reported POSIX subsystem that was "never really completed" [S15].

3

Linux had no choice

It "had to make the kernel boundary itself the stable interface" [V38].

[S15] *The sad history of the Microsoft POSIX subsystem* — Reiter (2010). <https://brian-reiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/> [S19] *Inside Native Applications* — Russinovich (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>

The previous slide called the placement a decision. That needs more care, because the evidence supports two readings, and one reading alone would overstate it [S15] [S19].

Reading A is design. NT had three operating environments: POSIX, OS/2 and Win32 [S19]. Each one served its own clients through its own interface. Each one reached NT through the native API [S19].

The source does not say where those environments ran, so this slide does not say either. On that reading the split is deliberate. An unstable lower layer then follows from it [inference].

Reading B is vestige. A POSIX subsystem did exist, and one practitioner account says it was built to satisfy United States government procurement rules and never finished [S15]. On that reading the split is what remained after the plan was abandoned.

Both accounts are in the literature, and a practicum does not settle them. The mechanism is settled. The intent is not.

Linux is the clearer case. It was a kernel built against a separately developed userland it did not control [V38]. It could not update the C library in lockstep, so it had to make the kernel boundary itself the stable one [V38].

33 33. Linux is the outlier, not Windows

On screen:

Part 4

Only Linux promises that the raw system call interface stays the same [V9]. Say different, never worse [inference].

1

Who makes no such promise

FreeBSD, OpenBSD, NetBSD, Solaris/illumos, macOS [V43].

2

Not promised is not broken

FreeBSD keeps old programs running through compatibility layers. It never writes the promise down [V36].

3

Why Windows and macOS can

They ship their own C library, so the library is the stable surface and the kernel boundary can move [V38].

Now the correction to the conclusion the last two slides invite.

Windows looks like the odd one out here. It is not. The source names the list: FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all behave like Windows [V43]. None of them promises that the raw system call interface stays the same. Linux does. Linux is the exception [V9].

That list is not five broken systems [V36]. FreeBSD keeps old programs running through compatibility layers. It holds the interface in practice. It simply does not write the promise down [V36].

Windows and macOS can skip the promise because each ships its own C library. Programs call the library, the library is the stable surface, and the kernel boundary below it is free to move [V38]. Linux could not do that, because somebody else built its C library.

In this course the platforms are different. Neither of them is worse. The word worse in an assignment gets one question back: which property does this rank them on?

34 34. One program, two kernels

On screen:

Part 4

One program, two systems, traced on one day.

STEP	LINUX [S37]	WINDOWS [S36]
<i>failed open</i>	<code>openat(... "no-such-file.txt" ...) = -1 EN-OENT</code>	<code>NtCreateFile(... "no-such-file.txt" ...) => 0xc0000034 [2 '...']</code>
<i>open</i>	<code>openat(... "iodemo.out" ...) = 3</code>	<code>NtCreateFile(... [0x124] ... "iodemo.out" ...) => 0</code>
<i>write</i>	<code>write(3, "one\ntwo\nthree\n", 14) = 14</code>	<code>NtWrite-File(FileHandle=0x124, ... Length=0xe ...) => 0</code>
<i>close</i>	<code>close(3) = 0</code>	<code>NtClose(Handle=0x124) => 0</code>

Same program, same fourteen bytes, one write on each side. Hold the program fixed, and what is left is the two systems [inference].

[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/iodemo-c/README.md

This is the slide the section was built for.

Read across the rows, not down. Each row is the same step of the same source file [S36] [S37].

Row one is the deliberate failure. The Linux call is `openat`, which is `open` with a starting directory [inference]. It returns `-1` and one error name [S37]. Windows returns a native code and the Win32 error it translates into [S36] [inference].

Row two is the open, and the two columns are not symmetric. Linux returns the file descriptor `3` as the return value [S37]. Windows returns the status `0` and hands the handle `0x124` back through an argument [S36] [inference].

Row three is the write. Fourteen bytes on Linux, `Length=0xe` on Windows [S36] [S37]. Three `fwrite` calls in the source, one call in each trace [inference].

One number to reconcile, from part two. Seven write operations there, one here. The other six were the Linux loader's [S37].

Now the caveat, and it is about the Windows tool rather than about this pair. NtTrace is one author's work [S10]. Microsoft documents those native entry points as subject to change between releases [S9]. So read the Windows column as the shape of the boundary, not as an interface you may depend on [inference].

The author's note on the earlier sample says "It is not a matched pair." [V23] This pair is matched. Both captures ship in this deck, and any student can rerun them.

35 35. What we have not looked at

On screen:

Part 5 · Buffering moves your errors

One question I will not answer, and the work that follows from it.

Six minutes left, and a list of topics is not worth them.

One question goes back on the screen, and it stays unanswered today. Two consequences of the stack follow it. The room already holds everything needed to see them.

36 36. `write()` returned success. The bytes are not on a disk.

On screen:

Part 5

I built the whole hour towards this sentence. I leave it unresolved.

The manual says it plainly: "A successful return from `write()` does not make any guarantee that data has been committed to disk" [S2].

Saltzer, Reed and Clark gave the general form in 1984. A lower level may offer "an incomplete version of the function... useful as a performance enhancement" [S11].

So when is it actually safe? That is the durability lecture's question. I leave it open today.

[S2] `write(2)` — Linux man-pages. [S11] End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984). <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

Read the first quote again. It comes from the manual page for the most basic `write` call there is. It says a successful return guarantees nothing about a disk [S2].

With the stack in hand that is obvious rather than alarming. A successful write means the bytes reached the next level down. Which level depends on which call the program made.

The second quote is here so the behaviour does not read as a bug awaiting a fix. Saltzer, Reed and Clark wrote it in 1984. A lower level may offer an incomplete version of a function as a performance improvement. The complete version needs the application at the end points. Every level in the diagram is an instance of that.

So: when is it actually safe? This slide leaves the question open on purpose. A programmer who believes it is already answered writes no durability step at all.

37 37. Buffering moves your errors

On screen:

Part 5

The error arrives somewhere other than where you caused it.

1

Why it moves

The bytes only reached a buffer [inference].

2

Where it arrives

Where the library flushes, so at fclose [inference].

3

The rule

Check the return value of the call that flushes [inference].

4

The idiom hides it

"the most common durability idiom in every mainstream language... contains no durability step anywhere" [V4].

Here is the practical consequence, and it costs a debugging afternoon at some point.

`fwrite` returned success because the bytes reached a buffer. A full disk, an unplugged device, an exceeded quota — no error from any of those has reached the library yet, and none can [inference].

That is why the demonstration program checks the return value of `fclose`. Almost no published C does. That single check is the difference between writing fourteen bytes and believing it.

The last row is worth reading slowly. The most common way to write a file in every mainstream language contains no durability step anywhere [V4].

Python differs here. A Python `with` block does raise if the close fails. The two languages differ in what they force the programmer to notice, and that is the property this comparison ranks them on [inference].

This is error reporting, not error recovery. Recovery belongs to the durability lecture.

38 38. Predict: all your writes succeeded — can the close still fail?

On screen:

Part 5

Yes or no, on paper. Then a second question: if it fails, is the descriptor still yours?

1

Every `write()` returned the byte count

The program did nothing wrong that it could see.

2

Now `close()`

Can it return -1? On what kind of filesystem?

3

If it did

Is the file descriptor number still open, closed, or unknowable?

Two answers, written down [S14].

Two predictions, both yes-or-no, both on paper.

First. Every write in the program returned the number of bytes requested. Can the close still fail? Commit to yes or no.

Second. Assume it did fail. Is the descriptor still open, already closed, or unknowable? One of the three, in writing.

These are not trick questions. Most of the room gets the first one wrong, and for a good reason. Nothing in the buffer stack on the earlier slide says that a close does any work [inference].

39 39. Why a close can fail when every write succeeded

On screen:

Part 5

The kernel deferred the write's error. The close surfaces it [S4].

1

Deferred

"errors on a previous write(2) operation are reported only on the final close()" [S4].

2

Where it bites

"silent loss of data ... with NFS and with disk quota" [S4].

3

The mirror

"A successful close does not guarantee that the data has been successfully saved to disk" [S4]. Part 6's question.

Against the stack: write() said done at level 3 [S2]. close() can still refuse [S4].

[S4] close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

So, the first answer: yes, it can fail, and the manual page tells you when [S4]. The write's error was deferred. The kernel took your bytes into the page cache and said done. Later it tried to push them on, and the network file system or the quota refused. Close is the last call in your program that can still report that.

The manual gives two examples, NFS and disk quota. In both the check happens after the write returned. Most of you will not hit them on a laptop. That is exactly why the habit of checking close matters. The code that works on your machine loses data on the server.

Read the mirror sentence too. A successful close does not mean the data is on disk. Close does not sync. So checking close is necessary and not sufficient, and Part 6 is about the sufficient part.

40 40. Observe: close it twice, and watch what the second close hits

On screen:

Part 5

`demo/close/` — one descriptor, two closes, both platforms [S39].

[code]

The first close released the number, error or not: Linux "always releases the file descriptor early in the close operation, freeing it for reuse" [S4]. So a retry closes nothing — or, with threads, "a reused file descriptor from another thread" [S4].

Under a debugger `CloseHandle` "will throw an exception if it receives ... a handle value that is not valid ... This can happen if you close a handle twice" [S38]. `NtTrace` is a debugger, so the second close raises [S39].

[S4] `close(2)`. <https://man7.org/linux/man-pages/man2/close.2.html> · [S38] `CloseHandle`. <https://learn.microsoft.com/en-us/windows/win32/api/handleapi/nf-handleapi-closehandle> · [S39] `demo/close/README.md`. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/close/README.md>

Now the second answer, and this one is observable. A ten-line program opens a file, writes, closes, and closes the same descriptor number again. That second close is exactly what a retry would do.

On Linux the trace shows it. Close of three returns zero. Close of three again returns minus one, bad file descriptor [S39].

The kernel released the number in the first call. The manual says it always does that early, and that the steps which can fail happen afterwards [S4]. So a failed close leaves you nothing to retry. Another thread may already hold that number.

On Windows the second CloseHandle fails with error six, invalid handle [S39]. Here is the twist. Microsoft documents that under a debugger the same call throws an exception instead [S38]. NtTrace is a debugger. So the traced Windows run raises rather than failing quietly. Both platforms report a double close as an error, and one of them raises.

Put the two answers together and the rule from three slides back follows. Check the return value of close. Never retry it. Treat a failure as data you may have lost.

41 41. Never retry a failed close

On screen:

Part 5

The obvious repair is wrong, and the manual page says so [S4].

"Retrying the close() after a failure return is the wrong thing to do, since this may cause a reused file descriptor from another thread to be closed" [S4].

A retry closes whatever now sits at that number, which may be another thread's file [S4].

Report the error. Then stop. Part 6 shows the same rule one level down, at fsync, and the data loss that taught it [V40].

[S4] close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

Now you check the return value of close. You will want to do the obvious thing with it, and the obvious thing is wrong [S4].

Do not retry. The manual page says why, and the reason is specific. After a failed close the descriptor may already be gone. If another thread has opened a file meanwhile it may have been given that same number. Your retry closes somebody else's file.

The transferable rule: when a failure leaves you unable to tell what state you are in, stop the program and report it. Continuing hides the damage and does not repair it.

The databases learned this the hard way, one level down, at fsync. That story has its own slides in Part 6, after you have seen what fsync promises [V40].

42 42. Networking and IPC are this same mechanism

On screen:

Part 5

Not new subjects. The same objects and the same calls, wearing different names.

1

A socket on Linux

It is a file descriptor, and Linux makes that contract binding [V28].

2

A socket on Windows

An opaque SOCKET, and Windows makes Winsock the contract [V28].

3

A pipe

A file descriptor too, so the same calls reach it [V29].

Depth belongs to the next lecture. I want to stop you filing networking under "something else".

Thirty seconds on this slide, and no more names after it.

Networking later in this course is not a new mechanism. On Linux a socket is a file descriptor. The same read and write calls work on it. Sockets, pipes and terminals are all file descriptors, which is why one program can wait on all of them at once [V29].

On Windows a socket is an opaque type rather than a file descriptor. The binding contract is Winsock, the Windows sockets interface, rather than the file-descriptor contract [V28]. That is section five's pattern in an unexpected place [V8]. Same job, different mechanism.

43 43. Honest limits

On screen:

Part 5

What today's tools cannot see.

-

The device level

[V5]

Not observable from software: no trace can check it.

-

Asynchronous I/O

[V11]

The tracer sees the setup, then no I/O [inference].

Here is what this lecture did not establish.

The device level is not observable from software [V5]. Section three said so, and this slide repeats it. It is the limit on today's thesis, and it is where data is actually lost [V5].

Asynchronous I/O breaks the model the room has just learned, and it breaks it in an interesting way. A modern Linux interface moves submission into shared memory rings [V11]. One option starts a kernel thread. That thread polls the rings.

Steady-state submission then makes no system calls at all. The tracer sees the setup and then nothing. The boundary just learned is already moving.

44 44. Honest limits, continued

On screen:

Part 5

And what today's tools cost.

-

strace in production

[V25]

Even the cheaper --seccomp-bpf mode [V39] "should therefore not be used on production systems!"

-

Mapping, crash dumps

—

Each has its own lecture.

And the tools. The tracer has a second and cheaper mode, selected with the option `--seccomp-bpf`. It hands the kernel a filter, so the tracer stops only for the requested calls. On the workload of rebuilding `strace` itself, plain `strace` costs 3.06 times the untraced run, and that mode costs 1.44 times [V39]. A `strace` contributor disqualifies it anyway.

The mode "still stops at each syscall of interest and should therefore not be used on production systems!" [V25]. Production uses a different mechanism, in-kernel programs rather than a stop at every call: eBPF on Linux, and event tracing on Windows. No per-call figure for either is available here, so this deck gives none. Today's tools watch the layer this lecture teaches, and that is the reason for them.

Finally, memory mapping and crash dumps. Each has a lecture.

45 45. Lecture 2 ends here. The question does not.

On screen:

Part 5

`write()` returned success and the bytes are not on a disk [S2] — so when is it safe? Part 6 measures it.

Part 5 named the question and the honest limits of the tools. Your prediction for Part 6 has been on your page since Part 1: under a second, about five, about thirty, or not until shutdown [inference].

That was the end of Lecture 2. It left one question open, and the room predicted an answer to it an hour ago. A successful write is no guarantee that the data is on disk [S2]. Check the page. Part 6 runs the program, and the room watches the kernel finish the job.

46 46. So when is it safe?

On screen:

Part 6

The question I left you with last week. Your prediction is already on your page.

Part 6. This is the part the whole lecture builds towards.

Lecture 2 ended on a question and left it unanswered. Write returned success and the bytes are not on a disk, so when is it actually safe?

Every student wrote a prediction an hour ago. Under a second, about five seconds, about thirty seconds, or not until shutdown. No changes now.

Before any run, take three answers from the room. Each student reads the prediction out and gives a reason.

Then the measurement runs on this machine, and the room sees which prediction holds.

47 47. The kernel's own count of what it has not written

On screen:

Part 6 · Durability

No tracer here. The kernel publishes the number itself, and we watch it fall.

[code]

Dirty

File data the kernel holds and has not written to the device yet. A page in this state is a dirty page [V78].

Writeback

Data the kernel is writing to the device right now [V78].

It is a machine-wide counter, so we watch it rise when `save.py` runs and fall when the kernel is finished [S52].

[S52] demo/durability/README.md — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

This demonstration uses no tracer at all, and that is deliberate.

Lecture 2 established that a tracer changes the program it watches. So this measurement does not watch the program. It watches the kernel, and the kernel publishes the number itself in the file `/proc/meminfo`.

Two fields matter. `Dirty` is file data the kernel holds and has not written to the device. `Writeback` is data on its way out right now [V78].

The word for a page in that state is a dirty page. It is the kernel's own word, and it appears in the output.

One honest caution before the run. This counter belongs to the whole machine, not to one file. Another program can write during the window. So watch the shape rather than the absolute number. It rises when `save.py` runs, and it falls when the kernel is done.

Run it now.

48 48. Observe: the program finished at zero seconds

On screen:

Part 6 · Durability

[code]

There it is. Read it from the top.

The first two lines are the kernel's own timers, printed before the run. The slide after next returns to them, because they are the interesting part.

Then the samples. At time zero the machine holds about one mebibyte of dirty data. That is the baseline.

Still at time zero, the figure jumps by four mebibytes. Those are the hundred records. Read the annotation on that line carefully. `save.py` has already exited. The process is gone. It printed done and returned to the shell, and four mebibytes of data sit in the kernel's memory.

Now read the next three lines. Half a second. Two and a half seconds. Four seconds. The number does not fall. It creeps up slightly, because the machine is doing other work.

Five seconds. It drops back to baseline. The kernel has written the bytes.

Five seconds after the program said done [S52]. Take a show of hands for five seconds, then for thirty.

One point on that last line, because it does not match what the watcher promises. The watcher samples until the count returns to its starting value. It finished at six hundred and seventy-six kilobytes, against a baseline of one thousand one hundred and seventy-

six [S52]. It went below. That is the counter being machine-wide rather than file-specific: another program finished its own writeback during the same five seconds. Read the fall, not the final figure.

49 49. Five runs, three sizes

On screen:

Part 6 · Durability

One run is an anecdote. Here are all five, including the one that disagrees.

WRITTEN	RUNS	THE KERNEL'S DONE, AFTER THE PROGRAM'S
$100 \times 40 \text{ KiB} = 4 \text{ MiB}$	3 (two on overlays, one on plain ext4)	5.0 s, 5.5 s, 5.5 s
$100 \times 64 \text{ B} = 6 \text{ KiB}$	1	5.5 s
$1000 \times 40 \text{ KiB} = 40 \text{ MiB}$	1	0.5 s

Four of the five runs land at five seconds, and file size barely matters between six kilobytes and four mebibytes [S52]. The forty-mebibyte run went out ten times sooner, which the next slide explains.

[S52] demo/durability/README.md — five runs, captured 2026-09-20. <https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

The program ran five times before this lecture, and the table shows all five [S52].

Four of them land in the same place. Five seconds, five and a half, five and a half, five and a half. The file size made almost no difference between six kibibytes and four mebibytes. That is the result.

The fifth run is the one worth attention. Forty mebibytes went out in half a second. Ten times sooner, for ten times the data.

One conclusion is tempting here and wrong: more data is not safer. The next slide gives the reason.

Two limits on this table. Every figure is one machine, one kernel, one file system. Two of the three big runs use the container's overlay file system, not a plain ext4 partition [S52]. An overlay file system stacks on top of another file system. One run repeated on plain ext4 gave the same answer [S52], and that repeat is why the number is on a slide.

50 50. Five seconds in memory that belongs to nobody

On screen:

Part 6 · Durability

The program's done and the kernel's done are different events, five seconds apart.

1

Where the bytes were

In the page cache, level 3 of your drawing. That memory is the kernel's [V78].

2

What set the delay

A kernel timer, not your program and not Python [V78].

3

What a power cut takes

Everything in that window [V14].

The return value was correct. It reported that the kernel had accepted the bytes [V50].

Now the explanation, and it is short, because the model is already in place.

Where were the bytes for those five seconds? In the page cache. Level three of the diagram drawn at the start of the hour. That memory belongs to the kernel and to no process, which is exactly why killing the process did not lose it [V78].

What set the five seconds? Not the program. Not Python. Not the C library. A kernel timer did, under a rule the program cannot see and did not set [V78].

This is also the answer to the third recap question. That question asked what a power cut loses. The window now has a size. On this machine, for an ordinary file, about five seconds [V14].

Read the last line out loud. The return value was correct. Write returns when the kernel has accepted responsibility for the bytes, not when the work has happened [V50]. It reported a handoff, and the handoff reads like a report about the disk.

That is the whole lecture, at Part 6 of ten.

51 51. Which rule fired, and how we know

On screen:

Part 6 · Durability

The setting a student reads first is not the one that applied.

dirty_expire_centisecs

This machine: 3000, so 30 s [S52]. It marks when data is "eligible for writeout" [S68].

dirty_writeback_centisecs

This machine: 500, so 5 s [S52]. It sets how often the "kernel flusher threads ... wake up" [S68].

The 40 MiB run

Above dirty_background_ratio, "writeback starts sooner" [V48].

Writeback is the kernel copying dirty pages out. Which rule fired here is my reading [inference].

[S68] [admin-guide/sysctl/vm.rst](https://www.kernel.org/doc/html/latest/admin-guide/sysctl/vm.rst). <https://www.kernel.org/doc/html/latest/admin-guide/sysctl/vm.html> [S52] [demo/durability/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md). <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

This slide states what the measurement does not prove.

A student who wants to know when writeback happens searches, and finds `dirty_expire_centisecs` first. The kernel page says it sets when dirty data is old enough for writeout [S68]. This machine read three thousand, which is thirty seconds [S52]. The pages went out in five. So the setting a student meets first is not the one that applied.

The same machine read the second setting as five hundred, which is five seconds [S52]. The page says this setting controls how often the flusher threads wake up [S68]. Five seconds is also the measured answer. That is a strong match.

The forty-mebibyte run went out before any timer, because it crossed a memory threshold. Above `dirty_background_ratio`, writeback starts sooner [V48]. That is why more data went out faster.

Here is the honest sentence. Writeback has several competing rules. The rule named here is inferred from the timing, and this demonstration cannot show the rule directly [inference]. Certainty needs a kernel tracer, and that is a different course.

Read both settings on your own machine before arguing with the number. Run `cat /proc/sys/vm/dirty_writeback_centisecs`. Then run `cat /proc/sys/vm/dirty_expire_centisecs` [S68].

The lesson survives either way. The kernel's done is on the kernel's clock, and no program sets it.

52 52. Visibility is not durability

On screen:

Part 6 · Durability

The objection you are about to make, and the answer to it.

Visibility

After a successful write(), a later read returns the bytes. Another process reading the file sees them too. This is a coherence property of the page cache [V45].

Durability

The bytes survive a power loss. This is a property of the medium [V45].

My own note calls confusing the two "the single most common error in this area" [V45].

So `cat results.csv` returning your data proves visibility. It proves nothing about the disk [inference].

One objection arrives here: the file opens and reads back, so it must be saved.

The read-back is real. POSIX guarantees it. After a successful write, a later read returns the bytes, and a read from a different process returns them too [V45].

That property has a name. It is visibility, and it is a coherence property of the page cache. The bytes sit in the kernel's memory, and the kernel serves the read from there.

Durability is a different property. It means the bytes survive a power loss, and it is a property of the medium [V45].

The reading note calls confusing the two the single most common error in this whole area [V45].

So here is the test that does not work. Running `cat` on the file proves visibility [inference]. It says nothing about whether the disk holds the data. During those five seconds, `cat` would have shown every byte.

Two words, held apart. Visibility. Durability.

53 53. Predict: what does one `fsync` per record cost?

On screen:

Part 6 · Durability

Before the counts. Same program, same hundred records, four ways [S14].

Measured

One `f.flush()` per record. 101 write calls, 1.1 ms of wall time [S52].

Your prediction

Add one `os.fsync()` per record — the third arrow on the stack you drew, level 3 to level 4.

How long?

Write a number in milliseconds. Not a factor, a number.

[S52] [demo/durability/README.md](https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md) — four-mode counts, 2026-09-20. <https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

One more prediction, and this one is quick.

The same program ran four ways. One of those ways flushes after every record. That pushes each record out of the Python buffer with a `write` system call. A hundred records, a hundred and one `write` calls, one point one milliseconds of wall time [S52].

Now the question. One line changes. Each record now also calls `os.fsync` after it flushes. That is the third arrow on the diagram drawn this morning, from the page cache down to the device. It asks the kernel to push that record to the device before returning.

Same hundred records. Same hundred and one `write` calls. The only difference is a hundred `fsync` calls on top.

How long? A number in milliseconds, not a multiplier. The point is to commit to a magnitude.

Ten seconds. Write it down.

Then take a show of hands for under five milliseconds, under twenty, and under a hundred.

54 54. Observe: sixty times, for the same hundred writes

On screen:

Part 6 · Durability

MODE	WRITE CALLS	FSYNC CALLS	WALL TIME
<i>default</i>	6	0	0.2 ms
<i>flush per record</i>	101	0	1.1 ms
<i>fsync per record</i>	101	100	67.3 ms
<i>unbuffered</i>	101	0	1.0 ms

Sixty times the wall time, for the same hundred write calls plus a hundred fsync calls [S52]. The cost is the waiting, not the extra crossings [inference].

Microsoft concedes the same: FlushFileBuffers "can be inefficient when used after every write to a disk drive device" [S40]. Most programs never pay it.

[S52] demo/durability/README.md — four-mode counts, 2026-09-20. <https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md> [S40] FlushFileBuffers function (fileapi.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers>

Sixty-seven milliseconds [S52]. Against one point one.

Read the table across. The write count is identical in three of the four rows. A hundred and one calls. What changed is a hundred extra calls, and each one waits for the device.

Notice where the cost is not. A hundred extra crossings did not cost sixty-six milliseconds. Lecture 2 measured a crossing in microseconds. The cost is the waiting, not the crossing.

This is also not a Linux quirk. Microsoft's own documentation for the Windows equivalent concedes the same thing about the same pattern. `FlushFileBuffers` can be inefficient when used after every write [S40]. That is the vendor writing about its own function.

Here is one reading of that gap, and it is only a reading. A text editor does not pay this cost on every keystroke. A database does. Neither program was measured here.

One number in passing, and a later slide returns to it. The tracer reported two point two milliseconds inside `fsync` against sixty-seven milliseconds of wall time. The other sixty-five were the process waiting for the device. Count under the tracer. Time without it. Lecture 2 stated that rule, and this is the instance.

55 55. What `fsync()` promises, and what it does not

On screen:

Part 6 · Durability

It crosses one boundary. The next is not yours to check.

1

Does

Transfers the file's data "to the disk device", "including ... flushing a disk cache" [S67].

2

Waits for

It "blocks until the device reports that the transfer has completed" [S67].

3

Cannot check

Whether the device flushed. No trace tells "an honest flush from a fast lie" [V5].

Durability is a chain: each call flushes across one boundary, and each can fail alone [V44]. The device reports. [T1]

[S67] `fsync(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/fsync.2.html>

Now the call itself, in the manual's own words.

`fsync` transfers the file's modified data to the disk device [S67]. The page says this includes writing through or flushing a disk cache, if the device has one [S67]. Good. That is level three to level four, and it is the intended effect.

It blocks until the device reports that the transfer has completed [S67]. Also good. It waits.

Read the last four words again. The device reports. The whole guarantee rests on that report being accurate, and Lecture 2 established that no software in this course can check it [V5].

The sentence at the bottom comes from the author's reading note, and it is worth keeping [V44]. Durability is not a property of a write. It is a property of a chain of calls. Each call flushes across exactly one boundary. Each one can fail on its own.

This is not a theoretical worry about the last step. SQLite's documentation records that "often the IDE disk control lies and says that data has reached oxide while it is still held only in the volatile control cache" [V86].

So `fsync` is not the fix. It is one crossing in a chain, and it is the last one a program can see.

56 56. Windows: the same shape, and now measured

On screen:

Part 6 · Durability

The Windows twin of the four-mode run, on my Windows 11 VM [S52].

<i>IOCost.EXE , 100 × 4 KIB</i>	<i>KERNEL WRITE OPS</i>	<i>WALL</i>
<i>WriteFile into the cache</i>	100	0.7 ms
<i>FlushFileBuffers per record</i>	100	27.9 ms
<i>FILE_FLAG_WRITE_THROUGH</i>	100	25.2 ms
<i>FILE_FLAG_NO_BUFFERING</i>	100	9.9 ms

Forty times, for asking the device to confirm each record [S52]. Linux's figure was sixty. The two ratios are not comparable: these records are 4 KiB, the Linux ones were 64 B [S52].

Now read the count column: 100 in every row.

[S52] demo/durability/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md> [S40] FlushFileBuffers — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers>

Many students in this room use Windows, so this slide is a measurement and not a paragraph of documentation. The program is `iocost.exe`. It is the twin of `save.py`: a hundred records, four modes, timed with `QueryPerformanceCounter` and counted with `GetProcessIoCounters`, which is Lecture 2's counter [S52].

Flushing after every record cost twenty-seven point nine milliseconds against nought point seven [S52]. Forty times. On Linux the ratio was sixty. Do not divide one by the other.

These records are four kibibytes, and the Linux records were sixty-four bytes [S52]. The shape is the same on both platforms. The two ratios are not comparable [inference]. Microsoft's own page concedes the cost for its own function [S40].

Now the column worth an argument. The kernel counted a hundred write operations in all four modes, including the two that go straight past the cache. Two lectures have said to count crossings. Here the count is the one number that says nothing, and the wall time is the number that does. The count says how many times the program asked. The time says who waited [inference].

A trace of the same program, filtered to three calls, shows the rest. Flush mode: a hundred and one `NtWriteFile` and a hundred `NtFlushBuffersFile`. Default mode: the same writes, and nothing flushed [S52]. The hundred and first write is the done line going to the console.

Three limits. The record sizes differ from the Linux run, as the slide says [S52]. This is an x64 binary under emulation on an ARM64 virtual machine [S52]. Read the shape, not the milliseconds. And under the tracer the nought point seven became twenty-six point six [S52]. Count with a trace, time without one, on Windows too.

57 57. The two kernels write in opposite orders

On screen:

Part 6 · Durability

The same watch, on Windows, until the counter returned to baseline [S52].

WRITTEN	WINDOWS CACHE MAN- AGER	LINUX KERNEL
4 MiB, three runs	0.5 s · 1.0 s · 0.5 s	5.0 s · 5.5 s · 5.5 s
40 MiB, one run	14.1 s	0.5 s

*Windows' **lazy writer** had the small file out inside a second. Linux waited for its flusher's five-second wake. On the large file they swap — Linux wrote it at once, Windows trickled 512 pages a pass [S52].*

Both clocks are a kernel's. Neither is your program's [inference].

[S52] demo/durability/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

Same demonstration, same four options on the page, other platform. Windows has no `/proc/meminfo`. So `watch_dirty_win32.py` samples the Cache Manager's own performance counters instead: Cache Dirty Pages, and Lazy Write Flushes per second [S52].

Read the table across. The four options covered both platforms, for different files. Under a second was the Windows answer for four mebibytes. About five seconds was the Linux answer for the same file. Then the forty-mebibyte run swaps them over.

So the lesson is not that one kernel is quick and the other slow. Each kernel has its own rules, and neither rule lives in the program.

One honest note on the big run, in the same shape as the Linux honesty slide. The pages went out in steady passes of about five hundred and twelve pages. The passes came roughly every second and a half. A final burst then finished the file. Which Cache Manager rule produced that pattern is a reading of the timeline, not something this demonstration shows [inference]. Windows Internals is the book for it, and a paraphrase here would not be a fact.

What the room did watch is this. The program said done, and fourteen seconds later the kernel finished.

58 58. Predict: your `fsync()` failed — what does the retry return?

On screen:

Part 6 · Durability

The device rejected the write, so `fsync()` returned `-1` with `EIO`. The obvious repair: call it again.

1

The second call

`-1` again · `0`, bytes on the device · `0`, bytes gone.

2

Who found out

A database team, in 2018, in production [S41].

One of three, written down next to your Part 1 prediction [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> · [S41] `fsyncgate`. <https://danluu.com/fsyncgate/>

Part 5 gave you a rule with no story behind it: never retry a failed close [S4]. Here is the story, one level down, and it starts with a prediction.

Picture the situation. Your program called `fsync`. The device reported a failure, so `fsync` returned minus one with `EIO`. Every instinct you have says try again. Transient errors are what retry loops are for.

Three options for what the second call returns. Minus one again, which is what a retry loop expects. Zero, with the bytes safely on the device, which is what a retry loop hopes for. Or zero, with the bytes gone.

Write one down. In a moment you see the trace, produced by the engineer who reproduced this after a database lost data in production [S41].

59 59. Observe: the retry returns success, over data that was never written

On screen:

Part 6 · Durability

2018, and the case is known as fsync-gate. A PostgreSQL developer found that retrying a failed fsync() "is not OK at least on Linux" [S41].

[code]

That is a real reproduction against an error-injecting device. Its author annotated the last line "fsync on file with still-dirty data+EIO returns success BAD", for a byte that "certainly never got written" [V53].

[S41] fsyncgate — the pgsql-hackers thread, archived. <https://danluu.com/fsyncgate/>

Read the trace. Lecture 2 read one line of the same shape of output [V53].

Line one. `fsync` fails and reports an I O error. That is correct behaviour. Something went wrong with the device. Note the boundary the person who found this drew: retrying is not OK "at least on Linux" [S41]. They did not claim it for every system, and two slides on the reason appears.

Line four. The program calls again, as almost any program would, and gets zero. Success. Over data that was never written to anything.

The person who reproduced this annotated that line: `fsync` on a file with still-dirty data returns success, and that is bad [V53].

Now check the prediction. "Minus one again" disagrees with the kernel. "Zero and the bytes are on the device" disagrees with the byte.

60 60. Explain: the retry does not retry

On screen:

Part 6 · Durability

Mechanical, not sloppy code: the retry does not re-attempt the failed I/O [V54].

1. **Reported once.** The kernel marks the page failed, reports it to the first `fsync()`, and clears the mark [S41].
2. **Already gone.** A failed writeback drops the data and marks the page clean, so a retry has nothing to write [V54].
3. **So the retry lies.** "It reads a status register that the previous read destroyed, over a page cache the data has already left" [V89].

The result: "how PostgreSQL lost data silently for twenty years" [V90].

Now why. And the reason is not that somebody wrote sloppy code. The retry does not re-attempt the I/O at all [V54].

Step one. The kernel tried to write the page, the device said no, and the kernel marked the page as failed. The first `fsync` call collected that mark and reported the error. Collecting it cleared it [S41].

Step two. The kernel also dropped the page. A failed writeback discards the data and marks the page clean, because the kernel will not keep unwritable memory around forever [V54]. So the bytes left the page cache when the first call failed.

Step three. The second `fsync` finds a clean page and no error mark. It returns zero. It is telling the truth about the state it can see, and that state has no memory of the failure. "It reads a status register that the previous read destroyed, over a page cache the data has already left" [V89].

So the obvious repair makes the failure silent. A failure that is both silent and cleared by the retry is the hardest kind to find. This one is "how PostgreSQL lost data silently for twenty years" [V90].

Notice the shape. Part 5 carried the slide "Never retry a failed close". This is the same rule, one level further down. Next: what the databases did about it.

61 61. What the engines did instead

On screen:

Part 6 · Durability

Three storage engines, three architectures, one answer, in one year.

PostgreSQL

[V40]

Retry replaced by a crash [V93].

2018

InnoDB

[V84]

Retry loop became a fatal error.

2018

WiredTiger

[V24]

No retry. Panic on a failed flush.

2018

A stopgap that "has now lasted eight years" [V56].

So what does a program do when a call can report success over lost data?

Three shipped storage engines answered in 2018, and they answered the same way [V40]. The program crashes.

PostgreSQL replaced the retry with a panic and back-patched it to every supported release [V93]. InnoDB deleted a sleep-and-retry loop and substituted a fatal error [V84]. WiredTiger converted its sync paths to panic, and wrote the reason into the source comment [V24].

The argument is worth a sentence, because crash on error sounds like giving up. It is not. A database writes every change to a log first, then later writes the data files and deletes the old log. That second step is a checkpoint, and it is the moment the database throws away its own last copy.

The `fsync` return code is the only evidence that lets it do that. Once that evidence may be false, the safe move is to discard the in-memory model and rebuild it from the log. Crashing is what preserves the log.

Two honesty notes. These are not three independent discoveries. WiredTiger's comment quotes the same LWN report the other two were reading [V24].

And this was a stopgap. The reading note records that it has now lasted eight years [V56], which is a polite way of saying nobody found anything better.

62 62. Windows here is not "the same", and not "safe"

On screen:

Part 6 · Durability

It is undocumented, which is a third answer and a worse one.

•

What is missing

[V47]

The PostgreSQL wiki's matrix of post-fsync-failure behaviour has no Windows row at all.

•

Why nobody can settle it

[V55]

The Linux contract was inspectable, so it was discoverable. The Windows contract "cannot be settled by reading" [V55].

Linux documents a hostile contract: one error report, then the data is gone. That is actionable. "For a system that must not lose data, an unspecified contract is the worse of the two positions" [V91].

Now the Windows half of fsync-gate. It is the clearest case in this course of reading what a specification does not say.

Eight years after this was found on Linux, the PostgreSQL project maintains a table of how each operating system behaves after a failed fsync. Linux, four kernel ranges. FreeBSD. NetBSD. macOS. Illumos. DragonflyBSD.

There is no Windows row [V47]. Not a row saying unknown. No row.

Why has nobody filled it in? On Linux the logic lives in shared kernel code that anyone can read. On Windows, `FlushFileBuffers` is a thin wrapper that dispatches to whichever filesystem driver owns the volume [V83]. The answer can therefore differ per driver, and the reading note says the Windows contract cannot be settled by reading [V55].

So the correct sentence for an exam is neither "Windows is safe" nor "Windows has the same bug". It is this: Linux documents a hostile contract, and the Windows one has never been settled [V55].

And for a system that must not lose data, an unspecified contract is the worse of the two [V91]. Reading what a specification does not say is a professional skill, and this is the clearest case in the course.

63 63. Sync the directory too

On screen:

Part 6 · Durability

This part is yours, and the classic recipe is incomplete [V51].

*A file is two pieces of data in two places: its **bytes**, and its **name**, which lives in the directory. `fsync()` on the file flushes the bytes. The manual: "Calling `fsync()` does not necessarily ensure that the entry in the directory containing the file has also reached disk" [S67].*

So "create-write-fsync-close on a new file is the classic incomplete recipe" [V51]: after a crash the bytes are safe and the file may have no name.

The fix costs one line: open the directory, and `fsync()` that descriptor too [S67].

Five slides covered things a program cannot control. This slide covers the part it can, and it is the part real code gets wrong.

A file is two things. The bytes, and the name. They are stored in different places. The bytes belong to the file. The name is an entry in another file, the directory.

A call to `fsync` on the file flushes the bytes. That call does not touch the directory. The manual page says so plainly, and almost nobody reads that far [S67].

So the recipe everybody writes — create the file, write, `fsync`, close — is incomplete [V51]. After a crash a file can have all its bytes and no name. It is unfindable.

The fix is one more call. Open the directory itself, which returns a descriptor. Then call `fsync` on that descriptor too [S67]. Now the name is on the device as well.

64 64. Write-then-rename, and why it needs a sync

On screen:

Part 6 · Durability

How every editor saves, and how ext4 made it lose files in 2009 [S44].

[code]

Why the pattern. `rename()` swaps the name atomically: "there is no point at which another process attempting to access `newpath` will find it missing" [S70].

The trap. The rename is a small piece of metadata. The temporary file's bytes are still dirty in the page cache. ext4 wrote the rename first, so a crash meant "that existing file gets replaced with a zero-length file" [S44].

The fix. `fsync()` the temporary file before the rename [S44]. Then the name can only point at bytes already on the device [inference].

Third, and this is the one that hit real users. Write a temporary file, close it, rename it over the original. That is how every editor saves, and there is a good reason for it.

The reason is rename. Rename replaces the name in one step. The manual promises there is no moment at which somebody opening the file finds it missing [S70]. So a reader gets either the whole old file or the whole new file. Never a half-written one. That is what the pattern buys you.

Now the trap. Rename changes a directory entry. That is a tiny piece of metadata, and filesystems write metadata eagerly. The bytes of the temporary file are a different matter.

They are sitting in the page cache, level three on your drawing, waiting for the write-back you measured in this part. The ext4 filesystem in 2009 wrote the rename to disk before the bytes. If the machine crashed in between, the original file was already replaced, by a file with a name and no bytes. Ts'o wrote that up himself, about a filesystem he maintains [S44].

The fix is the same call as before. `fsync` the temporary file, then `rename` [S44]. After that, the name can only point at bytes that already reached the device.

So the summary of this whole part is not give up [inference]. The device you cannot check. The directory you can, and it costs you one line.

65 65. The respectable opposite view

On screen:

Part 6 · Durability

Most programs are right not to pay, until the failure is "silent" [V52].

1

*Most need not pay**SQLite's own write-ahead-log setting "does lose durability", yet "a good choice for most applications" [S43].*

2

*Trusting the layer**Trusting the layer below is "a defensible ... position", and "the implicit default" [V52].*

3

*Blame the API**"no one ever, ever wrote creat(); write(); close(); rename(); and hoped they would get an empty file" [S45].**[S43] SQLite. https://www.sqlite.org/pragmas.html#pragma_synchronous [S45] Aurora, LWN (2009). <https://lwn.net/Articles/351422/>*

This part would teach badly if it left the room thinking every program must call `fsync`. Most should not.

SQLite is the cleanest example, because it documents the trade instead of hiding it. Its recommended mode in write-ahead logging does lose durability, and it says so in those words. It still recommends the setting for most applications [S43]. A transaction can roll back after a power cut. For a phone address book that is the right trade.

The reading note concedes the general position. Trusting the abstraction and covering the gap operationally is "a defensible engineering position for a great many systems", and it is "the implicit default" [V52].

The strongest version of the objection is Valerie Aurora's, and it argues against the position this part teaches. Nobody ever wrote that four-call sequence hoping for an empty file [S45]. On that reading, a system that demands this knowledge from every programmer has already failed.

So where does the defensible position stop being defensible? The note names it exactly. It fails where the failure is silent [V52]. Backups do not help when nothing reported the loss.

That is the line. Not "always `fsync`". Know when done arrived, and know whether losing it would be loud or quiet.

66 66. Log first — and what if the log write fails?

On screen:

Part 6 · Durability

The write-ahead log protects you because of an order, and the order covers its own failure [inference].

1. **Order.** *Log first, `fsync` ed. Only then committed [V85]. The data file follows later, at a checkpoint [S71].*
2. **Log `fsync` fails:** *nothing was promised yet. PostgreSQL panics here too: "we already did the same for WAL data" [S71].*
3. **Data-file `fsync` fails later:** *"the only remaining copy of the data is in the WAL" [S71]. Crash, replay.*

The log wins only because it is made durable first, one `fsync` per commit — the 67 ms you saw [S52].

A fair question, and one you should ask of every log-based design: the log only helps if it wins over the file. So what if the log write fails and the file write does not?

The answer is an order, and the order is the whole trick. Nothing is written to the data file until the same change is in the log and the log has been `fsync`d. And nothing is told to the client until that `fsync` returns. Committed means the log is on the device [V85].

Now run the failure through that order. The log's `fsync` fails. At that moment no client has been told committed. PostgreSQL treats that exactly like the data-file case. It panics, and the 2018 commit says so in one sentence: they already did the same for WAL data [S71]. When it comes back up, the transaction simply never happened, and no one was promised otherwise.

The other way round, the data file fails later, at a checkpoint. The log still has the change. The commit message says it: the only remaining copy of the data is in the WAL [S71]. Crash, replay, done.

So the log is not magically more durable than the file. It is made durable first, and paid for with one `fsync` per commit. That is the sixty-seven milliseconds you measured earlier in this part [S52]. Databases pay it. Your editor does not, and Part 6 is the reason [inference].

67 67. So when is it safe?

On screen:

Part 6 · Durability

The answer, and what it does not cover.

In software

The last done you see is `fsync()`, resting on the device's report [S67].

In hardware

A tested UPS keeps the host up. Level 4 is inside the drive, and needs its own power-loss protection [inference].

Neither covers

`fsync-gate` lost data with the power on [V53]. No battery helps [inference].

So also

Log every change first: "recover from the WAL after any failure" [V85].

SQLite trusts neither call — "neither ... works as advertised" [V49].

So. When is it safe.

In software, never completely. Every done in the stack is a handoff to the next layer. The last handoff a program can watch is `fsync`, and `fsync` trusts the device [S67] [V5].

In hardware, the honest answer arrived in Lecture 2 without a slide. The only real answer to durability, and it is still best effort, is a well-tested uninterruptible power supply. Keep the power on long enough for the last handoff to finish.

That sentence needs one correction, and it comes from the diagram drawn this morning. A UPS keeps the host running. Level four is the cache inside the drive. A drive can lose power while the host still has some. So a battery upstairs does not defend it [inference].

What defends level four is capacitors in the drive — power-loss protection — which is a datasheet item rather than an assumption. So the honest pair is a tested UPS above and a checked drive below. Durability is the conjunction of every crossing succeeding, and each crossing can fail on its own [V44].

Then the part neither of them covers. `fsync-gate` lost data with the power on and the machine running [V53]. A battery is no defence against a failure the kernel reports once and cannot report again [V54].

So the full answer has two halves. Keep the power on. And write every change to a log before writing the data. PostgreSQL's own documentation puts it plainly: "the only way to avoid data loss is to recover from the WAL after any failure is reported" [V85].

SQLite says it most plainly of anyone. It calls both primitives and trusts neither, because it has "received reports that neither of these interfaces works as advertised on many systems" [V49]. Its correctness rests on journalling and checksums instead.

That is the state of the art. Not a better call. A design that survives the call being wrong.

68 68. Lecture 4 starts here

On screen:

Parts 7 to 9 · The announced tail

Parts 7, 8 and 9 are built in full. If the clock has gone, Lecture 4 opens on this slide and we jump to part 10.

Thirty seconds on how the rest of today works.

Parts 1 to 6 and Part 10 were the plan for this slot. On time, the lecture carries on into the tail from here. Behind time, it skips to Part 10 now, and Lecture 4 opens on this exact slide.

Nothing is cut either way. All three parts are written, every demonstration in them is captured, the handout carries them, and the containers are in the repository today.

Announcing this beats quietly running out of time, for the same reason every measurement here states what it does not prove. A room should always know which part of a plan it is in.

Parts 7 and 8 are also the two topics Lecture 2 named and deferred, so reaching them closes two loops at once. Part 9 is the one topic today that Lecture 2 never mentioned.

69 69. What a pipe is

On screen:

Part 7 · Waiting

The thing we are about to wait on, a hundred times over.

"Pipes and FIFOs (also known as named pipes) provide a unidirectional interprocess communication channel. A pipe has a read end and a write end" [S59].

- *A pipe is a 64 KiB buffer inside the kernel [S59]. One process writes into one end, another reads from the other.*
- *Both ends are file descriptors, so `read()` and `write()` work on them unchanged [S59].*
- *"If a process attempts to read from an empty pipe, then `read(2)` will block until data is available" [S59]. That word, block, is the next slide.*

Before the prediction, two words you need, because the whole of Part 7 is built on them. First word: pipe.

A pipe is a buffer that lives inside the kernel. It has two ends. One process writes into the write end, another process reads from the read end, and the bytes pass through the kernel's buffer in between. On Linux that buffer is sixteen pages, sixty-four kibibytes [S59]. When you type `ls`, a vertical bar, `wc` in a shell, the shell creates a pipe and connects the two programs to its two ends.

Both ends are ordinary file descriptors. That is why `read` and `write` work on a pipe without any change in your program [S59]. Part 8 makes that the whole point of the lecture's second half.

And one sentence from the manual that sets up everything that follows. If a process attempts to read from an empty pipe, `read` will block until data is available [S59]. Block. That is the second word.

70 70. Blocking, non-blocking, and letting the kernel wait for you

On screen:

Part 7 · Waiting

Three ways to ask a pipe "is there anything to read?" [inference]

1. **Blocking.** `read()` on an empty pipe parks your thread until data arrives [S59].
Fine for one pipe, hopeless for a hundred.
2. **Non-blocking.** Set `O_NONBLOCK`. Now `read()` returns at once, with `EAGAIN`, when there is nothing [S59]. Ask all hundred, again and again: polling.

3. **Let the kernel wait.** `select()` and `poll()` take a list of descriptors and sleep until one is "ready", meaning a read "without blocking" is possible [S47]. One sleeping call instead of thousands of asks. `epoll` is Linux's faster form of the same idea [S46].

Second word: blocking. And the two things people built to avoid it.

Blocking is the default. You call read on an empty pipe and the kernel parks your thread. It does not run again until data arrives [S59]. For one pipe that is perfect, you use no processor while you wait. For a hundred pipes it is a problem. You asked pipe number one first, so you are asleep on pipe number one while pipe fifty has data waiting.

The first way out is non-blocking mode. You set a flag on the descriptor, `O_NONBLOCK`, and read stops waiting. If there is nothing, it returns immediately with an error called `EAGAIN`, try again [S59]. Now you can ask all hundred pipes in a loop. And then loop again. That is polling, and Program A in a moment does exactly this.

The second way out is to let the kernel do the waiting. The calls `select` and `poll` take a whole list of descriptors. They sleep until at least one descriptor is ready. Ready has a precise meaning in the manual, a read on it would not block [S47]. One call, one sleep, instead of thousands of questions.

The `epoll` interface is Linux's newer and faster version of that idea, and it keeps the list inside the kernel [S46]. Program B uses it.

Hold those three. Ask and wait. Ask and return. Let the kernel wake you. Now the prediction.

71 71. Predict: a hundred pipes, and nobody writing

On screen:

Part 7 · Waiting

Two programs, two seconds, zero bytes of data. They differ only in how they wait [S14].

The setup

A hundred pipes. Nobody ever writes to them. Both programs wait two seconds, then exit.

Program A

Asks every pipe in turn, non-blocking, round after round.

Program B

Registers all hundred once, then makes one call that sleeps.

The question

How many system calls does each one make? About a hundred · a thousand · thirty thousand · millions.

Two numbers, on paper. Neither program transfers a single byte.

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Part seven, and the tail starts here.

Two Python programs. Both create a hundred pipes that nobody ever writes to. Both wait two seconds. Both then exit having received nothing.

The only difference is how they wait.

Program A asks every pipe in turn. Is there anything? No. Next pipe. Is there anything? No. Round after round, for two seconds. The reads are non-blocking, so each one returns immediately with nothing.

Program B registers all hundred pipes with the kernel once. It then makes a single call. That call sleeps until something arrives or the two seconds are up.

How many system calls does each make? Pick from about a hundred, a thousand, thirty thousand, or millions. Two numbers.

One hint before the count. No data ever arrives. Every call either program makes buys the same information, which is that nothing has happened. Each model pays its cost while nothing is happening [V61].

Thirty seconds.

72 72. Where the question comes from

On screen:

Part 7 · Waiting

The cost of waiting, not of I/O.

Kegel, 1999

[S55]

"So hardware is no longer the bottleneck."

Named

Gammo and colleagues

[S49]

Plus 10,000 idle connections: "degrade by up to 79%".

Measured

*Up to 79% of `select / poll` throughput gone, only idle connections added [V61]. No answer made a `read` faster: each changed **"where the interest set lives"** [V62].*

Kegel's 1999 revision lacks the idleness clause everyone quotes [S55] [V63].

[S55] The C10K problem, 8 May 1999 — Kegel. <https://web.archive.org/web/19990508164301/http://www.kegel.com/c10k.html> [S49] Gammo et al. <https://www.kernel.org/doc/ols/2004/ols2004v1-pages-215-226.pdf>

Before the run, the origin of this question.

In 1999 Dan Kegel named the C10K problem. Ten thousand simultaneous clients on one machine. The opening line disposes of the explanation everyone reaches for first. Hardware is no longer the bottleneck. That sentence is in the earliest archived revision of the page, 8 May 1999 [S55].

So what was the bottleneck? Not moving bytes for the busy connections. Waiting on the idle ones.

Five years later a group at Waterloo measured it, and the design of the experiment is what makes it trustworthy [S49]. They held the request workload fixed. Same requests, same bytes, same server. Then they added ten thousand connections that do nothing at all.

Throughput fell by up to seventy-nine per cent [S49]. Nothing was added except idleness [V61].

Notice what none of the answers did. `epoll` did not make a read faster. Every one of the answers changed where the interest set lives and who re-states it [V62].

Now a precision point, and it matters because the Kegel page is easy to find. The famous sentence about most descriptors being idle is not in the 8 May 1999 revision [S55]. It appears in a later revision [S48]. The reading note calls the modern framing a sharpening of the original rather than a quotation of it [V63].

So cite the revision, or better, cite the measurement. The measurement is the thing that settles it.

73 73. A thread each, and why that stopped working

On screen:

Part 7 · Waiting

A hundred pipes, a hundred blocking threads — thread-per-connection — failed on two bills [S59].

1

Memory

Kegel priced the stacks: 2 MB each, "not an uncommon default value", against 1 GB of user-accessible address space. 512 threads, then no virtual memory left [V94].

2

Scheduler

Banga, Mogul and Druschel priced the rest: it "scales badly as the number of threads increases" [V95].

Either way the bill is per connection, not per byte [V96]. My note calls that "the same mistake at two altitudes" — my reading, not either source's [V96].

One design comes to mind first. A hundred pipes, so start a hundred threads, each blocking on its own pipe.

That design has a name: thread-per-connection. It failed on two bills.

The first is memory. A thread needs a stack. Two megabytes each was, in Kegel's words, not an uncommon default. A 32-bit process with one gigabyte of user address space runs out at five hundred and twelve threads [V94].

The second is the scheduler. The costs of scheduling, context switching and thread-state storage all grow with the thread count [V95].

Both bills are charged per connection, not per byte [V96]. That reading belongs to the author's note, not to either source [V96]. This is history and arithmetic, not a measurement taken today [inference].

74 74. Observe: two seconds, zero bytes, counted

On screen:

Part 7 · Waiting

[code]

28,017

read calls, 28,000 of them failing with EAGAIN [S53]

1

epoll_pwait call, which slept for the whole two seconds [S53]

EAGAIN is not a fault. It is the kernel answering "nothing yet" to a non-blocking read, and strace -c files it under errors [S53].

[S53] demo/waiting/README.md — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>

Twenty-eight thousand and seventeen against one [S53].

Read the left column. Twenty-eight thousand reads, and twenty-eight thousand of them returned an error. Two seconds of work to learn that nothing happened.

Read the right column. A hundred epoll control calls, which are the registrations, paid once. Then one wait call.

One thing to notice about the percentage column, because it does not say what you expect. It is processor time inside the kernel, and it is independent of wall-clock time [S1]. The single wait call shows under two per cent because a sleeping call costs almost no processor time. It still occupied the whole two seconds.

Hands up who wrote thirty thousand for the first. Hands up who wrote about a hundred for the second.

Now one word you need, because it will mislead you. EAGAIN. The tracer prints it in a column headed errors, and it is not an error [S53]. It is the kernel answering a non-blocking read with nothing yet. The polling program is working exactly as written. Twenty-eight thousand correct answers to a question that had no news in it.

You will meet a strace summary with a big errors column. Check what the errors actually are first. Only then conclude that something is broken.

75 75. What those counts do and do not say

On screen:

Part 7 · Waiting

Three things about 28,017 and 1 before you carry them anywhere [S53].

1. **28,017 grows with time.** Twenty seconds of polling makes ten times the calls, and every one of them learns nothing [S53].
2. **1 does not.** The `epoll` program paid a hundred registrations once, then one wait, however long it waits [S53].
3. **28,017 is too low.** The tracer slows every call. Untraced, the same loop "asked 1.3 million times in one second" [S53].

So: count under the tracer. Time without it [S1]. Brendan Gregg measured a loop of nothing but system calls running 442× slower under `strace` [S66] [V17].

Three things about those numbers before you carry them anywhere.

First, the twenty-eight thousand is a rate, not a total. Run the program for twenty seconds instead of two and it makes ten times as many calls, and every one of them learns nothing [S53]. That is the shape of the cost: it grows with time, not with data.

Second, the `epoll` program's one wait call is a constant. Run it for twenty seconds, or twenty minutes, and it still makes one. The hundred registrations were paid once. Two different curves, and that is the entire content of part seven.

Third, and this is the uncomfortable one. Twenty-eight thousand is a floor, not the real rate. The tracer slowed the loop down. Without the tracer, the same laptop asked one point three million times in one second [S53].

So here is the rule you take into the assignment. Count under the tracer. Time without it [S1].

Brendan Gregg's measurement of a loop that does nothing but system calls came out four hundred and forty-two times slower under `strace` [S66] [V17]. The tracer is not free, and it is least free exactly where you most want to count.

76 76. `epoll` is three calls

On screen:

Part 7 · Waiting

1. `epoll_create1` makes an **interest set** that lives in the kernel between calls [S46].
2. `epoll_ctl` adds or removes one descriptor. Our hundred registrations [S53].

3. `epoll_wait` sleeps until a registered descriptor is readable. Our one call [S53].

`select` and `poll` hand the kernel the whole list on every call [S46]. `epoll` states it once, which is why Program B's count was a constant.

Readable has a precise meaning: a read "without blocking" is possible, because at least one byte is waiting or the writer has closed its end [S47] [inference]. It does not say how many bytes.

The previous slide showed a trace of an API with no name yet. Here is the name and the three calls, so the room can state them rather than only recognise the shape.

Call one creates the interest set. The set lives in the kernel, and the call returns a descriptor that names it. That is the structural move: the set survives between calls, so nobody has to re-state it [S46].

Call two adds one descriptor to that set. In the demonstration program it ran a hundred times, once per pipe, and then never again [S53]. Registration is paid once, however long the wait then lasts.

Call three sleeps. It returns when one of the registered descriptors is readable, or when the timeout expires. In the demonstration program it ran once and slept for two seconds [S53].

The difference from `select` and `poll` is where the list lives. Those two take the whole list as an argument on every call, and the kernel walks it every time [S46]. `epoll` keeps the list.

One more word, because it is used loosely. Readable. The manual defines ready precisely: an operation, here a read, could be performed without blocking [S47]. On a pipe that means at least one byte is there. It can also mean the writer has closed, so a read returns end of file at once.

Readable says nothing about how many bytes [inference]. That matters on the next slide.

77 77. Level-triggered and edge-triggered

On screen:

Part 7 · Waiting

One flag. Get it wrong and a connection goes quiet [inference].

Level-triggered — the default

Unread data is reported on every wait. The manual: "a faster poll(2)" [S46].

Edge-triggered — EPOLLET

Reports only when something new arrives [S46]. Read 1 kB of a 2 kB arrival and the next wait "will probably hang" [S46].

The rule for EPOLLET : every time you are woken, read everything — until EAGAIN — because the kernel will not mention the rest again [S46]. Non-blocking descriptors, or that loop never ends [S46].

One design choice inside `epoll`. The flag is easy to meet, and the wrong guess is expensive.

Level-triggered is the default, and the demonstration program used it. While a descriptor still has unread data, every wait reports it again. The manual's own summary of that mode is reassuring. It calls level-triggered `epoll` a faster `poll`, usable wherever `poll` is used [S46].

Edge-triggered is the other mode, and the manual page's own worked example is the bug in miniature. A writer puts two kibibytes in a pipe. The wait reports the pipe. The program reads one kibibyte. The next wait probably hangs, with the other kibibyte still sitting there [S46]. The mode reports changes, and nothing changed.

So the rule, in one sentence. In edge-triggered mode, every wake-up must read all of the unread data. Call read until it answers `EAGAIN`, meaning nothing is left. Only then wait again [S46].

That loop ends only on a non-blocking descriptor. Otherwise the last read blocks instead of returning `EAGAIN` [S46]. A broken rule raises no error here. The connection simply goes quiet.

78 78. Three ways to wait, and we watch all three

On screen:

Part 7 · Waiting

One ladder. Each rung answers "who gets parked, and where" [V69].

1

Polling

The program asks each descriptor. Our 28,000 failed reads [S53].

2

*Readiness**The kernel says "you may read now" [S46]. Our one `epoll_pwait` [S53].*

3

*Completion**The program hands over a buffer. The kernel says "I have read for you" [V74].**In a trace: many failed reads means **polling**, one sleeping call means **readiness**, no reads at all during the wait means **completion** [inference].**[S46] `epoll(7)` — Linux man-pages. <https://man7.org/linux/man-pages/man7/epoll.7.html>*

Three ways, and both demonstration programs sat on the first two rungs.

Polling. The program asks. Every descriptor, every round, for as long as it waits. That was twenty-eight thousand reads [S53].

Readiness. The program registers its interest once and then makes one call that sleeps. The kernel wakes it and says: you may read now. The program still does the read itself [S46]. That was the `epoll` program.

Completion. The program hands the kernel a buffer and says: read into this. The kernel does the read and reports that it is done. On Windows this is overlapped I O and I O completion ports, and it is the native model there. A later slide in this part runs it on Windows, with a trace [S53].

The reading note frames all three as one question rather than a feature list: who gets parked, and where [V69].

Now the practical skill. Read a trace. Then name the model. All three models appear today [inference].

Many failed reads means polling. One sleeping call means readiness. A wait with no reads at all, and a completion dequeue instead, means completion [inference].

Notice also when each cost is paid. All of it is paid while nothing is happening.

79 79. Readiness is permission to try, not a promise

On screen:

*Part 7 · Waiting**The word ready sounds like a guarantee. It is not one.**select(2) , BUGS: it "may report a socket file descriptor as "ready for reading", while nevertheless a subsequent read blocks" [S47].**1**So use non-blocking mode**It is "safer to use O_NONBLOCK on sockets when blocking is undesirable" [S47].**2**The model**"permission to try, not a promise of success" [V58].**Ready reports a moment that has passed, on select and on epoll alike [V92]. A done, one rung up [inference].**[S47] select(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/select.2.html>*

One caveat that catches students and production code equally.

Ready does not mean the read will succeed. The manual page for `select` says so about itself, in its own bugs section. It may report a socket as ready for reading, and a later read still blocks [S47].

The mechanism: data arrives, the kernel marks the descriptor ready, and then a checksum error discards the data. The report was correct about a moment that has passed.

The consequence is a habit. Put the descriptor in non-blocking mode even in code that has just been told it is ready. The page recommends exactly that, calling it safer when blocking is undesirable [S47]. The same holds for `epoll` [V92].

The reading note carries the sentence worth keeping. A readiness notification is permission to try, not a promise of success [V58].

Notice what that is. It is today's thesis again, one rung up the ladder [inference]. The kernel said done about its own state at the instant of the call. The program reads it as done about its next call.

Same word. Different subject. Third time today.

80 80. The real difference is who owns the buffer

On screen:

Part 7 · Waiting

Not the API shape. The ownership, and it is why porting between the models is a re-write.

	READINESS	COMPLETION
<i>when you supply the buffer</i>	<i>after you learn the descriptor is ready [V75]</i>	<i>at submission, before any data exists [V74]</i>
<i>who owns it while waiting</i>	<i>you — the kernel holds no pointer [V75]</i>	<i>the kernel — you must not touch or free it [V74]</i>
<i>one buffer can</i>	<i>be shared across ten thousand connections [V75]</i>	<i>serve one outstanding operation [V74]</i>

My note calls ownership "the load-bearing row" [V57]. Microsoft's contract: clients "agree not to disturb the buffers in any way until the network has consumed the buffer's contents" [V74]. [T2]

One structural fact carries the whole of Part 7, and it is this one.

The two models differ in where the copy happens relative to the notification. Everything else follows from that.

In a readiness model, the program supplies the buffer after the kernel reports waiting data. The kernel holds no pointer into the process address space between notifications, so one scratch buffer may be shared across ten thousand connections [V75].

In a completion model, the buffer must exist before the data does [V74]. Each outstanding operation needs its own buffer [V74]. It belongs to the kernel until the completion packet arrives [V74]. Microsoft's own wording is that clients agree not to disturb the buffers until the network has consumed the contents [V74].

The reading note calls this the load-bearing row [V57]. Count the consequences. One buffer per outstanding operation, held for the whole wait. Ten thousand idle connections then carry a memory bill the readiness model does not have.

What completion buys is a genuinely zero-copy path and many reads in flight on one socket.

This is why changing function names does not port an event loop between the models. The note puts it exactly: code written for one model and mechanically ported to the other "is not slow — it is wrong" [V76].

81 81. It is not a Linux/Windows split

On screen:

Part 7 · Waiting

I have used the platforms as labels. Here is the correction.

1

The honest statement

"the readiness/completion split is not a Linux/Windows split" [V59].

2

The control case

BSD and macOS use kqueue, which is readiness [V81]. Not a Linux quirk [V59].

3

And both have crossed

Linux built a completion interface, Windows built a shared-memory ring [V68].

The useful question is not which system a program runs on. It is which done it is handed [inference].

This part has used readiness for Linux and completion for Windows as labels. The labels need a correction before an exam repeats them.

The reading note says it plainly. The readiness and completion split is not a Linux and Windows split [V59]. It is a split in the design space, and the two platforms entered it from different sides.

The control case is BSD and macOS. They classify with `epoll`: readiness notification, and the application does the I O [V81]. So readiness is not a Linux quirk. It is the Unix family's answer [V59].

Both platforms have since crossed. Linux, which had readiness for thirty years, built a completion interface. Windows has had completion since Windows NT, and it built a shared-memory ring instead [V68]. That is a different thing, and the end of this part returns to it.

So what is the accurate sentence? Not "Linux does readiness". Each platform's native model is the one its libraries and its documentation assume, and both models now exist on both systems.

The useful question about a program is not which system it runs on [inference]. It is which one it is handed. That is the question this hour opened with.

82 82. The trap this demonstration sets

On screen:

Part 7 · Waiting

We measured pipes. A regular file behaves differently, and the difference is silent.

•

Readiness and regular files

[V73]

epoll "has nothing useful to say about a regular file, which is always "ready"". The call returns at once and you learn nothing.

•

Why a completion model exists at all

[V88]

Linux "grew a completion interface because readiness could not express what its users needed".

So when you do meet these interfaces, do not expect a disk file in an `epoll` set to wait. It will not [inference].

One warning, because it is the first thing that catches a first event loop.

This part demonstrated readiness on pipes. Pipes are a good teaching case, because a pipe with no writer genuinely has nothing to give, and the kernel can say so.

A regular file on disk is different. `epoll` has nothing useful to say about it, because a regular file is always ready [V73]. The question "may this disk file be read" is answered yes, immediately, every time, whether or not the data is in memory.

The failure is silent. The loop raises no error. It spins, and it looks like a bug in the program.

That gap is not a curiosity. Linux grew a completion interface because readiness could not express what its users needed [V88]. The regular file is the case it could not express [V73].

So the sentence to write down is this. Readiness works on things that can be not ready [inference]. Sockets, pipes, terminals. Not files.

83 83. Four Windows words before the Windows pair

On screen:

Part 7 · Waiting

The same two programs, in Windows vocabulary [S53].

1. **Named pipe**. *"a named, one-way or duplex pipe for communication between the pipe server and one or more pipe clients" [S72]. A pipe with a name, openable by unrelated processes.*
2. **PeekNamedPipe**. *Looks without reading: it copies data "without removing it from the pipe" and "always returns immediately" [S73].*
3. **Overlapped ReadFile**. *A read that returns before the data exists. The kernel fills your buffer later [V74].*
4. **Completion port, GetQueuedCompletionStatus**. *A queue of finished operations. The call takes one, and "If there is no completion packet queued, the function waits" [S74].*

Four words, and then the Windows pair reads like the Linux pair.

A named pipe is the Windows pipe a program actually meets. Microsoft defines it as a named, one-way or duplex pipe between a pipe server and one or more clients [S72]. The name is the point: two unrelated processes can both open it by name, where a Linux pipe has to be inherited from a parent. The demonstration pipes are local, and nobody ever writes into them.

`PeekNamedPipe` is the polling call. It looks into the pipe without taking anything out, and the documentation says it always returns immediately, even when the pipe is empty [S73]. That is the exact counterpart of the non-blocking read on the last slide.

An overlapped `ReadFile` is a read that does not wait for the data. The program gives the kernel a buffer and a place to report back, and the call returns at once. The kernel fills the buffer when data arrives, later, while the program does other work. The reading note puts it precisely: the buffer must exist before the data does, and it belongs to the kernel until the completion arrives [V74].

The completion port is where the kernel reports back. It is a queue of finished operations. `GetQueuedCompletionStatus` takes one entry off that queue, and waits when the queue is empty [S74]. That is the one sleeping call, the twin of `epoll_wait`.

84 84. Predict: the same hundred pipes, on Windows

On screen:

Part 7 · Waiting

Completion is the rung we have not yet watched run [S14].

Program C

A hundred named pipes. PeekNamedPipe on each, round after round — the Windows polling twin.

Program D

One overlapped ReadFile posted per pipe, all bound to one completion port, then one GetQueuedCompletionStatus that sleeps.

The question

Under NtTrace, how many native calls does each make in two seconds — and how many reads does Program D do while it waits?

Two numbers again. Both programs are in `demo/waiting/` [S53].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S53] `demo/waiting/README.md`, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>

Part 2 set a standard: one program, both platforms, one capture day. Part 7 holds to it, so here is the Windows pair.

Program C is the polling twin. A hundred named pipes with connected clients that never write, and `PeekNamedPipe` on each one, round after round.

Program D is the completion program. Read its description carefully, because completion is the rung this part has only described so far. It posts one overlapped `ReadFile` per pipe, up front, before any data exists. It binds all hundred pipes to one completion port. Then it makes one call, `GetQueuedCompletionStatus`, and sleeps.

Notice what is strange there. The reads were already submitted. The kernel holds a hundred of the program's buffers while nothing happens. That is what handing over ownership means, and it is the row the ownership table called load-bearing [V57].

So: two numbers. How many native calls does each program make in two seconds? And the second question is the one that matters — how many reads does the completion program do while nothing arrives?

Write both down. The counts come from captured runs on a Windows 11 virtual machine, each one under NtTrace [S53].

85 85. Observe: Windows, traced, both ways

On screen:

Part 7 · Waiting

[code]

3

native calls per PeekNamedPipe: create an event, control the pipe, close [S53]

1

NtRemoveIoCompletion, which slept the whole two seconds [S53]

*Untraced, the polling program made **330,100** peeks in two seconds. Traced, it made 2,500 [S53]. Count under the tracer. Time without it.*

[S53] demo/waiting/README.md — NtTrace64 captures, Windows 11 build 26200, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>

Read the left column first. Two thousand five hundred peeks under the tracer. Every single one of them is three native calls, not one: create an event, do the pipe control, close the event [S53]. Seven and a half thousand crossings to learn nothing. The Win32 call you wrote is not the unit the kernel charges you for, which is exactly Lecture 2's lesson arriving in a new place.

Now the right column, and this is the shape worth recognising. A hundred `NtReadFile` calls, all at the start, before any data exists. A hundred calls binding each pipe to the port. One call to create the port. Then one `NtRemoveIoCompletion`, which slept for two seconds and came back with `STATUS_TIMEOUT` [S53].

Count the reads made while waiting. Zero. The reads happened before the wait, and the kernel owns a hundred of the program's buffers for the whole two seconds.

That is the third tell from the ladder, made visible. Many failed asks is polling. One sleeping call is readiness. Reads submitted up front and then a completion dequeue is completion.

Two honesty notes. The hundred errors `NtTrace` reports on `NtFsControlFile` in both traces are the `ConnectNamedPipe` calls returning `STATUS_PIPE_CONNECTED` [S53]. That is an informational status, not a failure. And the `PyWin32` pair shows the same shape one layer up, with the interpreter's own file reads mixed in [S53]. That is the reason to compare two runs rather than trust one.

86 86. Which numbers you may compare, and which you may not

On screen:

Part 7 · Waiting

Two counts per platform. Compare within a platform, never across [S53].

You may, and shall, compare within one platform, both runs traced:

- *Linux: 28,017 read calls against 1 `epoll_pwait` [S53].*
- *Windows: 8,153 native calls, three per peek, against 956, of which one `NtRemoveIoCompletion` slept the whole two seconds [S53].*

You may not compare 28,017 with 330,100, since one is traced and one is not [S53].

Nor Linux with Windows: different machines, and read is not `PeekNamedPipe` [S53].

What survives: the same shape, twice. Thousands of asks against one sleeping call, on two kernels that agree on little else [inference].

Four large numbers have now appeared in one part, and two of them invite a wrong division. So first, what may be compared, and what the assignment shall compare.

Within one platform, both runs traced the same way. On Linux, twenty-eight thousand reads against one `epoll wait` [S53]. On Windows, eight thousand native calls against nine hundred and fifty-six [S53]. The eight thousand are two and a half thousand peeks at three native calls each. One of the nine hundred and fifty-six slept the whole two seconds [S53].

Those pairs are honest. Same machine, same tracer, same clock.

Now what may not be done. Twenty-eight thousand and seventeen is a Linux count under `strace`. Three hundred and thirty thousand one hundred is a Windows count with no tracer at all [S53]. Those two do not sit on one axis.

Nor do Linux and Windows against each other. One is a container on one host, the other a virtual machine on another. A non-blocking read is also not a `PeekNamedPipe`.

So what survives the refusal? The shape, within each platform. Thousands of asks against one sleeping call. Same answer, twice, from two kernels that disagree about almost everything else [inference]. Comparing across two measurements is the most common way a correct pair of numbers turns into a wrong claim. Saying so now prevents it in the assignment.

87 87. What a ring is

On screen:

Part 7 · Waiting

One word before the last slide of Part 7.

A ring is two queues in memory that your program and the kernel both see: "ring buffers which are shared between user space and kernel space" [S75]. "You place I/O requests you want to make on the SQ, while the kernel places the results of those operations on the CQ" [S75].

*Why it matters here: every wait we counted today was a system call. With a ring you queue many requests and cross the boundary once [S75], or, with a kernel-side polling thread, "steady-state I/O submission needs **no syscalls at all**" [V11].*

One more word, because the last slide of this part uses it, and it should not sound like magic.

A ring is two queues, sitting in memory that both the program and the kernel can read and write [S75]. One queue is for requests. The program writes into it: read this, write that. The other queue is for results. The kernel writes into it: done, here is how many bytes [S75]. Neither side needs a system call to put something in its queue, because the memory is shared.

Why does that matter after this hour? Every wait counted today was a system call. Twenty-eight thousand of them in one program, one in the other. With a ring, a program puts many requests into the queue and crosses into the kernel once to say go [S75]. In one configuration a kernel thread watches the queue. Then there is no crossing at all, and the reading note says steady-state submission needs no syscalls at all [V11].

Keep the two ideas apart. The ring is how requests and results travel. Whether a readiness report or a completed operation travels is a separate question, and the next slide says which platform took which [inference].

88 88. Where both platforms ended up

On screen:

Part 7 · Waiting

Not convergence. Each one took the half it was missing.

Linux

Readiness-native. Built a completion interface, `io_uring`, in kernel 5.1 [V79].

Windows

Completion-native since NT [V80]. Built a ring, `IoRing`, in Windows 11 build 22000 [V80].

"Linux adopted completion semantics; Windows adopted the shared-memory transport" [V68]. The ring and the completion model are two separable ideas, and each platform took the one it lacked.

Learn IOCP, the I/O completion port, as current Windows practice. Rings are "the direction of travel", not the exam [V60].

Last slide of Part 7, and it is the satisfying one.

Linux was the readiness platform. Then it built a completion interface called `io_uring`, in kernel five point one [V79].

Windows has had overlapped I O and completion ports since NT, so it needed no completion model. What it lacked was the shared-memory ring, and it built one, called IoRing, in Windows eleven build twenty-two thousand [V80].

Now read the sentence carefully, because the tempting summary is wrong. This is not two platforms converging on the same thing. Linux adopted completion semantics. Windows adopted the shared-memory transport [V68].

The ring and the completion model are two separate ideas. They happened to ship together on Linux. Each platform took the half it was missing.

One instruction for this course, taken from the reading note. Teach I O completion ports as current Windows practice, and mention the ring as the direction of travel [V60]. On Linux, `epoll` is the equivalent present tense [inference].

And one honest limit to close on. A ring moves the system-call boundary, and the tracer sits on that boundary. With a kernel-side polling thread, steady-state submission makes no system calls at all [V11]. The tracer sees the setup and then nothing. The boundary Lecture 2 taught is already moving.

89 89. Same mechanism, different clothes

On screen:

Part 8

A file, a pipe and a socket. One program, one trace, three destinations.

Part 8. Its Linux result is a two-line comparison, and its Windows result is the opposite of that.

Lecture 2 ended by naming networking and interprocess communication as the same mechanism, and asked the room not to file them under something else. Today the trace shows it.

Three predictions and three runs. The first is a pipe. The second is a TCP socket, with netcat on both ends. The third is Windows, where the answer comes out the other way round [S57].

The program is the same `iodemo` from Part 3 today. Nine lines of C. It contains no code about pipes and no code about networks.

90 90. Predict: which of the four calls changes?

On screen:

Part 8 · Three destinations

You know the file run by heart now. I change only the destination [S14].

[code]

I now run `./iodemo /dev/stdout | cat`. The bytes go into a pipe and `cat` reads them.

On paper

Which of the four calls changes — the `openat`, the `write`, the `close`, or nothing?

And

Does the byte count in the `write` change?

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Here is the file run again. The room has read this trace three times today, so it needs no reading out.

Four calls. A failed open, a successful open, one write of fourteen bytes, a close.

Now one thing changes. The program takes its output path as an argument. The argument becomes `/dev/stdout`, and standard output goes into `cat`. So the same binary, with the same three `fwrite` calls, now writes into a pipe.

Two questions. First, which of the four calls changes? The `openat`, the `write`, the `close`, or nothing at all.

Second, does the byte count in the `write` change?

Before answering, consider which layer would have to change. The `fwrite` calls are in the C library. The `write` is in the kernel. Which of the two selects its behaviour from the destination? The library selects its buffering from the destination [V71].

Thirty seconds.

91 91. Observe: the file run and the pipe run

On screen:

*Part 8 · Three destinations**[code]**The same four calls, the same fourteen bytes in one write , the same descriptor number [S54]. cat on the other end received one two three .**The openat argument changed. The call did not, the count did not, and the byte total did not [S54].**[S54] demo/iodemo-c/README.md — the pipe run, captured 2026-09-20. <https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>*

Side by side. Read across the rows [S54].

Row one. Identical. The deliberate failure does not care.

Row two needs a correction to the question. The call is still `openat` , and it still returns descriptor three. The argument differs, because the run passed a different path. The answer "the `openat` changes" reads the argument, and it is right about what is on the screen.

So the question deserves a more precise form. Nothing changed about which calls were made, how many there were, or how many bytes each carried. Only the argument changed, and the command line changed it.

Row three is the result. One write. Fourteen bytes. Descriptor three. Exactly what the file run did.

Row four. Identical.

And cat received one, two, three. The bytes arrived.

Nothing in the program depends on what is on the other end, and nothing in its trace records what was there.

92 92. The descriptor is the abstraction, not the file

On screen:

*Part 8 · Three destinations**The reason is a kernel structure, not a convention.*

"On Linux, anything that takes a file descriptor takes a socket, because the kernel routes both through the same dispatch table" [V64]. A pipe too [V29]. [T3]

So `write(3, ...)` said done when the bytes were in the pipe. Had `cat` read them yet? Probably not — but the capture covers only the writer, so that is my reading [inference].

1

write call, for a file and a pipe alike [S54]

[S54] demo/iodemo-c/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>

Why does this work? Not by convention and not by luck.

The reading note on sockets puts it as a kernel structure. Anything that takes a file descriptor takes a socket, because the kernel routes both through the same dispatch table [V64]. A pipe is in that table too [V29].

So the descriptor is the abstraction. Not the file. The file is one of the things a descriptor can point at.

Now the thesis, one more time, in this new place. Write returned fourteen. What had happened at that moment? The bytes were in the pipe's buffer, in kernel memory.

Had `cat` read them? Probably not yet. This is the point to be careful, and the care is worth copying. The capture covers the writer only. It does not cover `cat`. So the claim that `cat` had not read them is a reading of how a pipe works, not something this trace shows [inference].

Same shape as Part 6. The call said done about a handoff. It said nothing about the other end.

93 93. Predict: the same program, writing to your terminal

On screen:

Part 8 · Three destinations

./iodemo /dev/stdout with nothing after it. No pipe, no file. The screen [S14].

*The same three `fwrite` calls: `"one\n"`, `"two\n"`, `"three\n"`. The file run and the pipe run each made **one** `write()` of fourteen bytes [S54].*

On paper: how many `write()` calls when the destination is a terminal — one, or three? And a second answer: who decides, the kernel or the C library?

One more destination, and the prediction comes first here, before the answer looks obvious.

Same binary. Same three `fwrite` calls — one, two, three — each with its newline. Writing to a file made one `write` of fourteen bytes. Writing into a pipe made one `write` of fourteen bytes [S54].

Now the run has nothing after it. Standard output is the terminal. The screen.

Two answers on paper. How many `write` system calls: one, or three? And which layer decided, the kernel or the C library? The second question is the one that matters, and the trace answers it [S14].

94 94. Observe: three writes, not one

On screen:

Part 8 · Three destinations

[code]

Same binary, same container, run with a terminal attached (`podman run -t`). Captured 2026-09-21 [S54]. Three system calls where the pipe run made one. The program did not change. The C library's decision did [inference].

Read it against the pipe run. Same `openat`, same descriptor three, same close. But three writes — four bytes, four bytes, six bytes — one per line [S54].

The only change is what sits on the other side of descriptor three. Not the program, not the kernel, not the bytes. A terminal instead of a pipe.

So the program made three system calls where it made one before, and nothing in its source code says so. Something between `fwrite` and `write` chose differently. That something is the C library, and the next slide carries its rule [inference].

95 95. Explain: the destination changed the trace, at the C library, not the kernel

On screen:

Part 8 · Three destinations

"Attached to a terminal, `stdout` is line-buffered... Attached to a pipe or a regular file, `stdout` is fully (block) buffered" [V71].

- 1. **The library asks once**, at the first write, what the descriptor is. A terminal gets line buffering: every `\n` becomes a `write()` [V71].*
- 2. **A file and a pipe** sit on the same side of that rule, so the first two runs agreed by construction [inference]. Both destinations are ones the library treats the same.*
- 3. **The kernel's part never changed:** `openat`, `write`, `close`. Only how many times [inference].*

Try it yourself: `./iodemo /dev/stdout` in a terminal, then with `| cat` [S54].

Here is the rule, and it belongs to the C library, not to the kernel. Attached to a terminal, standard output is line-buffered. Attached to a pipe or a regular file, it is fully buffered, in blocks [V71].

The library asks the kernel once, at the first write, what kind of thing sits behind the descriptor. A terminal switches it to line buffering, and from then on every newline pushes a write out. Three lines, three writes [V71].

Now look back at the two runs compared before this one. A file and a pipe are both on the block-buffered side of that rule. So they agreed by construction [inference]. That pair did not establish that the destination makes no difference. It used two destinations the library treats identically, and the slide says so.

Notice what did not change. The kernel's calls are the same three names, on the same descriptor. The destination changed the trace at the library, above the boundary, in how many times the same call was made. Below the boundary, nothing knew [inference].

Run it both ways, one after the other [S54]. It takes thirty seconds, and it sticks.

96 96. Predict: the network leg

On screen:

Part 8 · Three destinations

Same fourteen bytes, over a TCP socket, with netcat on both ends [S14].

The setup

./iodemo /dev/stdout | nc 127.0.0.1 9000, with nc -l listening. Both netcats traced.

The question

The listener receives the fourteen bytes from a TCP socket. Which call takes them off the wire?

Two options

read, the same call you would use for a file — or a socket-specific call.

One of two. Write it down.

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Second leg, and this one goes over the network.

One honest point about the setup first. `iodemo` cannot open a socket by path. A socket is not reachable through `/dev/stdout`, and that is itself a fact about the boundary [V65]. So the pipe carries the fourteen bytes to a netcat, and netcat puts them on the socket.

That makes netcat's traces the interesting ones, not `iodemo`'s. Both ends are captured.

The question. The listening netcat receives fourteen bytes from a TCP socket. Which system call takes them off the wire?

Option one. `read`. The same call a file would use.

Option two. A socket-specific call. Something called `recv`, or `recvfrom`, or something else entirely.

Most socket code in any language spells the call `recv` rather than `read`. So the question is whether that spelling says anything about the system call underneath, or only about the library on top.

Fifteen seconds. One of two.

97 97. Observe: both netcats, traced

On screen:

Part 8 · Three destinations

[code]

The client puts them on the wire with `write(3, ...)`. The listener takes them off with `read(4, ...)`. The same two calls `iodemo` used for a file and for a pipe [S54].

What is new is only how the descriptor came to exist [S54].

[S54] [demo/iodemo-c/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md) — the traced netcat pair, captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>

Both ends, side by side [S54].

Left column, the client. It creates a socket. It connects. It reads the fourteen bytes out of the pipe with `read` on descriptor zero. Then look at the next line.

`write`, descriptor three, fourteen bytes. That is the same call `iodemo` used for the file. It is the same call `iodemo` used for the pipe. Those bytes are now on a TCP connection.

Right column, the listener. Socket, bind, listen, accept. Four calls, and all four of them are about creating a descriptor. Then `read` on descriptor four, fourteen bytes.

`read`. Not `recv`. Not `recvfrom`. The same call a file would use.

Take a show of hands for `read`.

So here is the shape of the whole part. Look at the first four calls in each column. Those are all different, and they are all about how the descriptor came into existence. After that line, everything is `read` and `write` [S54].

One detail in passing, and it connects the two halves of the tail. The client's connect returned `EINPROGRESS`. This netcat opened its socket non-blocking and waited for readiness before writing. That is Part 7's mechanism, appearing in Part 8's trace without being asked.

98 98. Then what are `send` and `recv` for?

On screen:

*Part 8 · Three destinations**They are the program's choice of spelling, not the kernel's.**send(2) settles it in its own opening: "The only difference between send() and write() is the presence of flags. With a zero flags argument, send() is equivalent to write(2)" [S50].**1**Use them for a flag**MSG_DONTWAIT and MSG_NOSIGNAL have no write() equivalent [inference].**2**Otherwise it is style**They tell a human reader "this is a socket" [inference].**[S50] send(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/send.2.html>*

So if `read` and `write` work, why does every socket tutorial use `send` and `recv`?

The manual answers in its first paragraph, and the answer is shorter than the question. The only difference between `send` and `write` is the presence of flags. With a zero flags argument, `send` is equivalent to `write` [S50].

That is not a paraphrase. That is the page.

So when is `send` the right call? When the code needs a flag [inference]. There are several, and none of them has a `write` equivalent. Do not wait if there is no room. Do not raise a signal if the peer is gone. Those are real, and production code uses them.

The rest of the time, the choice is style [inference]. The word `send` in socket code tells a human reader that this descriptor is a socket. That is a legitimate reason, and it is a reason about readers, not about kernels.

What it is not is evidence of a different mechanism. The netcat in the capture used `read` and `write`, and the bytes crossed a TCP connection.

99 99. The same interface is not the same contract

On screen:

Part 8 · Three destinations

Three boundaries on the result.

-

One send is not one receive

[V70]

TCP is a byte stream. Marking where a message ends is your job — "the single most common practical bug in student socket code".

-

A size effect

[V82]

"writes of less than PIPE_BUF bytes must be atomic" — 4 KiB here [V67] [V82].

-

The default is not total

[V87]

"a strong default, not a total function" [V65]. lseek on a socket means nothing.

Before this part closes with "it is all the same", three ways it is not.

First, and this one costs a debugging night at some point. One send is not one receive. TCP is a byte stream. A program writes fourteen bytes, and the reader may get four then ten, or may get them glued to the next message.

Marking where one message ends is called framing. It belongs to the program, not to the kernel. The reading note calls it the single most common practical bug in student socket code [V70].

So the clean result of one write and one read is partly an effect of size. Fourteen bytes fit anywhere.

Second, the same holds for the pipe. POSIX says writes below PIPE_BUF must be atomic [V67], and that threshold is four kibibytes on Linux [V82]. Above it, two writers' records can be shredded into each other [V82]. Fourteen bytes were never going to show that.

Third. Everything is a file is a strong default, not a total function [V65]. Seek on a socket is meaningless, and the kernel marks sockets to skip the file-position locking they cannot use [V87].

So the accurate takeaway is this: the interface is shared, and the contract behind it is not. The calls transfer. The assumptions do not.

100 100. Predict: the same three destinations, on Windows

On screen:

Part 8 · Three destinations

On Linux the destination changed nothing in the trace. Windows is the contrast [S57].

1

Pipe

WriteFile to an anonymous pipe. Fourteen bytes.

2

Socket

send() on a loopback TCP socket, fourteen bytes.

3

The same socket again

WriteFile on that socket, cast to a HANDLE.

*On paper: **is Windows the same as Linux here: only the destination changes, the call stays the same? Yes or no, and if no, which leg differs?** One program, three destinations [S57].*

[S57] demo/destinations-win32/README.md — captured 2026-09-20. <https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/destinations-win32/README.md>

Many students in this room use Windows, and two demonstrations have just run on a platform where the destination is invisible. Here is the other platform, and it answers differently.

One program, three legs, the same fourteen bytes each time. Leg one writes to an anonymous pipe with `WriteFile`. Leg two sends on a loopback TCP socket with `send`. Leg three writes to that same socket, this time by casting it to a handle and calling `writeFile` on it [S57].

The prediction is not which native call each leg makes, because this API is new to most of the room. The prediction is whether the call stays the same when only the destination changes, as it did on Linux. If the call changes, which leg is the odd one out? The interesting half is leg one against leg three: same Win32 function, two different kinds of object. Does the kernel see one call or two?

Part 4 is the place to reason from. On Windows the stable boundary is the Win32 API, and the native layer underneath belongs to Microsoft. That cuts both ways here.

Thirty seconds. Both programs are in the folder, in C and in Python, and both captures come from the same virtual machine on the same day [S57].

101 101. Observe: the API changes the call, the object does not

On screen:

Part 8 · Three destinations

[code]

1 · pipe

[S57]

NtWriteFile, 14 bytes

file path

2 · send()

[S57]

NtDeviceIoControlFile, control code 0x1201f

device control

3 · WriteFile

[S57]

NtWriteFile — same handle as leg 2

file path

*Legs 2 and 3 are one handle, two native calls. A plain `writeFile` first failed with error 87 — Winsock sockets are **overlapped handles** [S57].*

[S57] *demo/destinations-win32/README.md*, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/destinations-win32/README.md>

Read the handles, because the handles are the argument.

Leg one, the pipe, is `NtWriteFile` with length fourteen. An ordinary file write.

Leg two, the send on the socket, is not a write at all. It is `NtDeviceIoControlFile` with control code one two zero one f. That code is the AFD send code, and `afd.sys` is the driver behind Winsock. The decoding comes from the sources, not from the tool: NtTrace has no AFD table and prints the number [S57].

Leg three is the one to stare at. Same handle as leg two, and `NtWriteFile`, the same native call as the pipe [S57]. So the file API reached the socket's file object directly.

Put those two facts together and the accurate sentence follows, and it is not the usual one. On Windows the chosen API changes the native call. The kernel object underneath is a file object either way. Linux showed the mirror image: one call, three destinations, nothing visible in the trace.

And one lesson from the first attempt, which failed. A plain `WriteFile` on the socket returned error eighty-seven, invalid parameter, because Winsock creates sockets as overlapped handles by default [S57]. Both programs now supply an `OVERLAPPED` structure and wait. So on Windows the file API on a socket is Part 7's completion model, asked for or not.

102 102. Windows: a `SOCKET` is not a `HANDLE`

On screen:

Part 8 · Three destinations

Third time today, and the same asymmetry each time.

Linux

A socket is a file descriptor, and Linux makes the file-descriptor contract binding [V28].

Windows

An opaque `SOCKET`, and Windows makes Winsock the contract [V28].

The NT kernel does hold a file object underneath [V28] — which is exactly what leg 3 just wrote to [S57]. The type is not a C-runtime file descriptor, while the handle is a kernel file handle [V66].

So the platforms agree on the mechanism and disagree on which layer they promise [inference]. Third time today.

Last content slide of Part 8, and it is the third appearance of one idea.

On Linux a socket is a file descriptor, and the file-descriptor contract is binding [V28].

On Windows it is an opaque `SOCKET` type, and the binding contract is Winsock [V28].

Now the careful version, because the tempting summary is wrong and a curious student will catch it.

One layer down, Windows does treat a socket as a file. Underneath Winsock, the NT kernel is "also holding a file object" [V28]. Leg three wrote to that file object with `NtWriteFile` [S57]. But the contract Windows publishes is Winsock, not the file object [V28]. That is why leg three needed an `OVERLAPPED` structure to work at all. No production code should depend on the file object.

The reading note reconciles the two Microsoft pages that look contradictory. The type is not a C runtime file descriptor. The handle is a kernel file handle [V66]. Both sentences are first-party, and both are true, about different layers.

So the sentence for an exam is this one. The platforms agree on the mechanism and disagree on which layer they promise [inference].

That idea has now appeared three times in one lecture. System calls in Part 4. Waiting in Part 7. Sockets here. It holds at three different layers, which is the reason for returning to it.

103 103. Bytes you never touch

On screen:

Part 9

One call can carry several buffers. And the kernel can move a file your process never holds.

Part 9, and it is the one topic today that Lecture 2 never mentioned.

So far every done in this lecture named a place the bytes are. This part covers two calls that change how the bytes get there. Both are best observed by what is missing from the trace.

Two demonstrations, two predictions, both in the container. The first is `writew`, and it removes crossings. The second is `sendfile`, and it removes copies. Those are two different savings, and this part keeps them apart, because collapsing them is the mistake it exists to prevent.

104 104. Predict: three buffers, one message

On screen:

Part 9 · Zero-copy

A message that is three pieces in your program, and one message in the file [S14].

The program

A header, a body and a trailer — three separate buffers. You want all three in the file, in order, as one message.

Question one

How many system calls does that take — three, one, or zero?

Question two

How many times does anything copy the three buffers to make it happen?

Two answers, written separately. They are not the same question [S56].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S56] [demo/scatter/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md) — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

Last prediction of the day, and it needs two answers rather than one [S14].

Here is the situation, and every student has met it. A program writes a record to a log, or a frame to a socket. Inside the program it is three separate things: a header, a body and a trailer. In the file it has to be one message, in that order, and nobody else's line may land in the middle of it.

The obvious way is three write calls. The other obvious way glues the three into one buffer first and makes one write call.

Question one: how many system calls. Three, one, or zero.

Question two, and this is the one that matters. How many times does anything copy those three buffers to make it happen? Count the copies inside the program, and the copies the kernel has to make.

Write both numbers. The two questions are separate because crossings and copies are two different costs, and almost every explanation of this topic runs them together.

Thirty seconds.

105 105. Observe: three lines become one

On screen:

Part 9 · Zero-copy

[code]

*Three crossings became one, and the return value is the whole message: 57 bytes [S56].
The three buffers never moved in the program.*

[S56] demo/scatter/README.md — `strace -e trace=write,writev`, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

Read the top block first. Three write calls, three crossings, and two lectures of reading lines exactly like these.

Now the bottom block. One call. The array inside the brackets is the argument: three entries, each with a base address and a length. That structure has a name, `iovec`, and its two fields are `iov_base` and `iov_len`, both visible on the screen. Three entries, and the count three after the closing bracket.

The return value is fifty-seven. Not eight. The call reports the whole message.

So question one is answered: one call, not three. The kernel gathered the three buffers, in order, in one crossing.

Now question two, and this is the place to check the written answer. How many copies did the program make? None. The three buffers stayed exactly where they were, and the kernel read them in order [S56].

Be careful with the word none, because the three-write version also copied nothing in the program. `writev` saved no copy. It saved two crossings. That is the next slide.

106 106. Gather buys crossings, not copies

On screen:

Part 9 · Zero-copy

The distinction the name hides, and your second answer.

What `writenv` removes

Two of the three user-to-kernel crossings, plus the staging buffer you would glue the pieces into [S58].

What it does not remove

Any copy the kernel makes further down. The four levels underneath are unchanged [S56].

`readv` is the mirror: one crossing scatters into several buffers [S58]. Valkey's reply path gathers framing, a stored string and a terminator into three `iovec` entries, then sends them with one `writenv` [S65]. [T4]

[S58] `writenv(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/writenv.2.html> [S56] demo/scatter/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md> [S65] `valkey/src/networking.c` — Valkey (2026). <https://github.com/valkey-io/valkey/blob/00a8a19554a39facb591837ea41ba4350260919d/src/networking.c>

This is the slide that separates the two answers, and it is the reason the prediction asked for two numbers.

`writenv` removes crossings. Three became one. That is a real saving, and it is the whole of what the call buys.

`writenv` does not remove a copy. Three plain write calls would copy nothing inside the program either. What `writenv` spares is the other design: glue the header, body and trailer into one new buffer and write that. That version really does copy, and it copies for no reason except the shape of `write`. Removing that staging buffer is the honest way to state the win.

Below the crossing, nothing changes. The same four levels drawn this morning sit underneath a `writenv` exactly as they sit under a `write`.

`readv` is the same idea on the read side. One crossing, and the kernel scatters the bytes into several buffers instead of one [S58].

And the use worth remembering is the boring one. Valkey's source reserves exactly three `iovec` entries for one reply element [S65]. They are the length prefix, the stored string itself, and the two-byte terminator. The comment in the code says why: use `writew` to save system calls and TCP packets [S65]. Not a headline optimisation. The removal of a buffer that only existed to satisfy the shape of an API.

107 107. Two promises, and only one of them is unconditional

On screen:

Part 9 · Zero-copy

Atomic means one thing: your bytes land as one unbroken block, with nobody else's write in the middle [S58].

- 1. **On a regular file** the promise is unconditional: "written as a single block that is not intermingled with output from writes in other processes" [S58]. A log line written with one `writew` stays whole.*
- 2. **On a pipe** it holds only below `PIPE_BUF`, 4096 bytes. Above that the kernel "may interleave the data" with another writer's [S59].*
- 3. **The trap:** the limit applies to the sum of your buffers, not the largest [inference]. Eight 1 KiB pieces are 8 KiB, and may interleave.*

The previous slide almost said "writew is atomic", and that would have been half right in the half that matters least. First the word, because it is used loosely. Atomic here means the bytes arrive as one unbroken block. No other write can land in the middle of them.

On a regular file that promise is unconditional. The data goes in as a single block, not intermingled with writes from other processes [S58]. That is why a log line written with one `writew` stays whole, however many processes write to the same log.

On a pipe or a socket the rule is different, and Part 8 already stated it. Writes below `PIPE_BUF` must be atomic. Above it the kernel may interleave the data with another writer's [S59]. On Linux `PIPE_BUF` is four thousand and ninety-six bytes.

And the trap. The limit is on the total, not on the largest piece [inference]. Eight one-kibibyte buffers in one `writew` are eight kibibytes, twice the limit. Two processes doing that into the same pipe can interleave, and each of them checked its own biggest piece and assumed safety.

108 108. Predict: moving a file you never look at

On screen:

Part 9 · Zero-copy

4 MiB from a file into a pipe. Two programs, same bytes [S14].

Program A

A copy loop: read 64 KiB into a buffer, write it out, repeat.

Program B

One call per chunk: `sendfile(out, in, ...)`. The program never allocates a buffer for the data.

The question

How many data system calls does each make — and how many times does each byte cross into your address space?

Neither program looks at a single byte of the file [S56].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S56] [demo/scatter/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md) — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

Second demonstration, and the same two-part question: calls, and copies [S14].

Program A is the loop every student has written. Read a chunk into a buffer, write the buffer out, go round again. Four mebibytes in sixty-four kibibyte chunks.

Program B replaces the pair with one call, `sendfile`. It takes a destination descriptor, a source descriptor and a length, and it moves the bytes. The program holds no buffer for the data at all.

The question has the same shape as before. How many data system calls does each make? And how many times does each byte cross the boundary into the process address space and back out?

The second number is the one that matters here, and it is why the two demonstrations stay apart. `writew` changed the first number. `sendfile` changes the second.

One thing to hold while predicting: neither program ever looks at the bytes. Not one line in either of them inspects the data. That is the precondition, and the end of this part returns to it.

109 109. Observe: half the calls, and no trip through your program

On screen:

Part 9 · Zero-copy

4 MIB INTO A PIPE	DATA SYSCALLS	COPIES THROUGH USER SPACE	WALL
copy loop, 64 KiB chunks	64 read + 64 write = 128	two per chunk	2.6 ms
sendfile	64 sendfile	none	2.0 ms

The trace's tell is an absence: with `sendfile` there is **no read line at all** [S56]. The bytes went from the page cache to the pipe without visiting your address space.

Sixty-four calls because each moved 64 KiB — the pipe's capacity, not a `sendfile` limit [S56] [S59].

[S56] [demo/scatter/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md) — `strace -c -e trace=read,write,sendfile`, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md> [S59] `pipe(7)` — Linux man-pages. <https://man7.org/linux/man-pages/man7/pipe.7.html>

Read the table across. A hundred and twenty-eight data calls against sixty-four, and two copies per chunk against none [S56].

The copies are the number that matters. In the loop, every chunk crosses into the process, sits in a buffer nobody reads, and crosses back out. Two copies, four mebibytes each way, to move bytes nobody inspected. `sendfile` takes a source, a destination and a length, and the bytes go from the page cache into the pipe [S56].

The tell in the trace is what is not there. No read lines. Part 7 had the same shape on the completion side: the evidence for a mechanism that removes work is an absence, not a presence.

Now the right question. Why sixty-four calls? The prediction offered three, one, or zero, and the answer is sixty-four. Each call moved sixty-four kibibytes, which is what the pipe accepts at once [S56]. A pipe holds sixteen pages [S59]. So the pipe's capacity set the chunk size, and `sendfile` has no such limit.

And one number this slide deliberately does not sell. Two point six milliseconds against two point zero, on one machine, once. That difference would not survive a second run's noise, and this deck does not claim it [S56]. The claim is the copies avoided and the calls halved. Those are counted, not timed.

110 110. What the name does not cover

On screen:

Part 9 · Zero-copy

It counts the copies it removes at one boundary. No others.

1

Direct memory access stays

The bytes still move. The CPU no longer carries them [S60].

2

It needs the page cache

Level 3 is not bypassed. Handing out its pages is the mechanism [inference].

3

It buys no durability

The same four levels sit below [S67].

4

It cannot help you look

Encrypt them and the process must hold them [S60].

[S60] sendfile(2). <https://man7.org/linux/man-pages/man2/sendfile.2.html> [S67]

fsync(2). <https://man7.org/linux/man-pages/man2/fsync.2.html>

Today's thesis applied to a word rather than to a return value. That is the reason this part is in the lecture at all.

Zero-copy counts copies across one boundary: into and out of the process address space. It says nothing about the others.

The two transfers at the ends remain. The disk still moves bytes into memory, and the device still takes them out. What the call removed is the CPU carrying them through the program in between.

The page cache is not bypassed. It is required. Sendfile works precisely by handing the destination references to pages the kernel already has [inference]. A file with no cached pages has nothing to reference [inference].

Here is the part worth carrying. The two most famous ways to make file I/O fast, sendfile and direct I/O, cancel each other. The nginx server encodes exactly that in its configuration: enabling directio automatically disables sendfile for that request [S61].

It does nothing for durability. The four levels are unchanged and a sendfile that returned is a handoff like every other done today [inference].

And the precondition, which is the honest limit. Zero-copy moves bytes the kernel already has to somewhere else the kernel controls. The moment a program has to compress, encrypt or template the bytes, it must hold them, and there is nothing left to remove [S60].

111 111. Zero-copy moves bytes faster. It does not make them safer.

On screen:

Part 9 · Zero-copy

Fewer copies changes who copies, not when the bytes are safe [inference].

- 1. **To a file.** sendfile copies "within the kernel" [S60], into the page cache. Nothing is on the device until fsync() [S67].*
- 2. **To a socket.** The return counts bytes the kernel accepted, not delivered: "No indication of failure to deliver is implicit in a send()" [S50].*
- 3. **One new duty.** The kernel may still be reading your pages after the call returns: keep the file "unmodified until the reader on the other end of out_fd has consumed the transferred data" [S60].*

Done still means accepted [V44].

Before you leave this part thinking zero-copy is a free upgrade, one slide on what it does not change: durability.

Zero-copy changes who does the copying, and how many times. It does not move the point at which the bytes are safe. Run it against the stack you drew this morning.

To a file. `sendfile` copies inside the kernel [S60], and where do the bytes land? In the page cache. Level three. Everything Part 6 told you still applies. The kernel's done is not the device's done, and nothing is on the medium until you `fsync` [S67].

To a socket. The return value counts the bytes the kernel accepted into its send buffer. It does not count bytes that reached the other machine. The manual for `send` says it flatly: no indication of failure to deliver is implicit in a `send` [S50].

And one thing zero-copy adds rather than removes. Because the kernel did not copy your data, it may still be reading your pages after the call has returned. The manual tells you to leave the transferred part of the file unmodified until the reader on the other end has consumed it [S60]. A copy would have freed you from that. Zero-copy does not.

So the sentence to take away is the one this whole lecture has been saying. Done means accepted by the next layer [V44]. Zero-copy makes the acceptance cheaper. It does not make it mean more.

112 112. Windows, and a silent fallback

On screen:

Part 9 · Zero-copy

Other names, and a Python fallback that says nothing.

1

Gather, a socket

WSASend with an array of WSABUF [S69].

2

Gather, a file

WriteFileGather, but every buffer must be at least an aligned page [S62].

3

Zero-copy

TransmitFile [S63].

Without `os.sendfile()`, `socket.sendfile()` uses "`send()` ... as a less efficient fallback" [S64]. No warning: on Windows you measure that [inference].

[S62] `WriteFileGather` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefilegather> [S63] `TransmitFile` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/mswsock/nf-mswsock-transmitfile> [S64] `socket` — Python docs. <https://docs.python.org/3/library/socket.html#socket.socket.sendfile>

This part carries no Windows capture, and the slide says so plainly: for Part 9, Windows is documentation. Parts 6, 7 and 8 each had a Windows pair with a trace. This one does not, and the evidence weighs accordingly.

The names first. For sockets the true counterpart of `writetv` is `WSASend`. It takes an array of `WSABUF` structures, each one a pointer and a length. The page says it allows multiple send buffers, which makes it applicable to the scatter gather type of I O [S69].

For files there is `WriteFileGather`. It is not the same tool wearing another name. Every buffer must be at least a system page, and page-aligned. The handle needs the flags `FILE_FLAG_OVERLAPPED` and `FILE_FLAG_NO_BUFFERING`, so it is an overlapped handle with no cache behind it [S62]. That is a database primitive, not a general one. One comparison is worth keeping, and it is `WSASend`.

`TransmitFile` is the zero-copy call: a file onto a socket without the round trip through the program [S63].

Now the part the Windows users in the room most need, because it is a trap. Python's `socket.sendfile` uses the real system call where it exists. Where it does not, on Windows, it quietly uses `send` instead, and the documentation says so in those words: a less efficient fallback [S64]. No error. No warning. The same line of Python.

So a student who runs `sock.sendfile` on Windows, measures it, and concludes that zero-copy does nothing has measured the fallback [inference]. That is the abstraction being honest about its interface and silent about its mechanism, which is the whole subject of this course. Anyone running that experiment this week checks which call actually ran before believing the number.

113 113. Homework, and the next question

On screen:

Part 10

What changed about the assignment, what you can run this week, and where Lecture 4 picks up.

Part 10. Four slides, and then the lecture ends.

Two things to settle. First, the assignment: Lecture 1 named a date, and that date has moved. Second, the question Lecture 4 opens on.

In between sits the voluntary work for this week.

114 114. Assignment 1 is not set today

On screen:

Part 10 · Close

I told you 22 September in Lecture 1. That has moved, and here is the honest reason.

What changed

The submission infrastructure is not ready. The assignments move back.

New dates

None yet. I will not put a date on a slide before I can keep it.

What it will measure

Buffered against unbuffered, and sequential against random access, over a 1 MiB file.

Predict the ratio. Measure it. Explain it in crossings.

Same method

Predict, observe, explain [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Lecture 1 set assignment one for the twenty-second of September. That is today, and the assignment is not being set.

The reason is administrative, and it sits on the teaching side rather than the student side. The submission infrastructure is not ready. Rather than set an assignment with no way to hand it in, the assignments move back.

There are no new dates yet. A date invented on a slide and corrected next week is worse than no date. When the infrastructure is ready, the dates follow, with the same two-week shape Lecture 1 described.

What the assignment asks for has not changed. Buffered against unbuffered. Sequential against random access. A one-megabyte file [S14].

Predict the ratio before any run. Measure it. Then explain the difference in operating-system crossings [S14].

That is exactly the work of the last two weeks in this room. Written predictions are already a rehearsal of the assignment.

115 115. Voluntary homework, and three rules for it

On screen:

Part 10 · Close

Everything I ran today ships with this deck. Run it yourself.

1

Predict first

Write the number first. Watching alone teaches nothing [S51].

2

Report version and buffer size

Python's default binary buffer moved from 8 KiB to 128 KiB [S7].

3

Count under the tracer

Then time it again, without the tracer [S1].

demo/durability/ · demo/waiting/ · demo/iodemo-c/ · demo/scatter/ · demo/close/ run in the container. demo/destinations-win32/ needs Windows [S57].

[S51] Crouch, Fagen, Callan and Mazur (2004). <https://works.swarthmore.edu/fac-physics/203/> [S7] CPython gh-117151. <https://github.com/python/cpython/issues/117151> [S1] strace(1). <https://man7.org/linux/man-pages/man1/strace.1.html>

Here is the work for this week, and it is voluntary.

Every demonstration from today ships with this deck. A Makefile and a Dockerfile in each folder, and a README recording the captured result. Run them. Then compare your numbers with the README.

Five of them run in the container on any machine. The Windows ones need a Windows machine, and those are the three most worth an independent check. The captures come from one virtual machine [S57]. The faculty machines are not that machine.

Three rules, and they are the same three Lecture 2 gave.

One. Predict first, on paper, before any run [S51]. The first three minutes of today argued against doing otherwise.

Two. Report the Python version and the buffer size beside every number. This is not bureaucracy. Python's default binary buffer moved from eight kilobytes to a hundred and twenty-eight [S7]. A disagreement with the numbers in this deck may be that alone, and then neither result is wrong.

Three. Count under the tracer. Then time the program without it. That is two runs [S1]. Today's waiting demonstration is the reason. The traced count was a floor, and the untraced rate on one laptop was two orders of magnitude higher [S53].

A number that contradicts this deck is a good week's work. Bring it.

116 116. Summary — what follows from this

On screen:

Part 10 · Close

1. **Acknowledgement is not durability** [T1] — a return value names the layer that took the bytes [V50].
2. **Readiness or completion** [T2] — one is permission to try [V58], one holds your buffer [V74].
3. **One mechanism, three destinations** [T3] — one `write` carried a file, a pipe and a socket [S54].
4. **Crossings are not copies** [T4] — `writew` removed crossings, `sendfile` removed copies [S56].

Name the boundary first: "I flushed" is ambiguous across four [V44]. Every number came from a run [S52] [S53] [S56]. Whose done did you measure? [V50]

Not a recap of the part titles. The four takeaways from the start of the hour come back here, each as a consequence.

One [T1]. Read a return value as a handoff to the next layer down. It names the layer that has taken responsibility for the bytes [V50]. It says nothing about the layer below that. Design for that done being wrong: three storage engines with three architectures reached the same answer [V41], and they replaced retry with crash and abort [V40]. Recovery then comes from the write-ahead log [V85].

Two [T2]. Readiness and completion are two answers to one question. Readiness is permission to try, and no more [V58]. Completion takes the buffer and keeps it until the packet arrives [V74]. Code ported mechanically between them "is not slow — it is wrong" [V76].

Three [T3]. One file descriptor, three destinations. The same `write` carried the bytes to a file, to a pipe and to a socket [S54].

Four [T4]. When something claims to be faster, ask what it removed. `Writev` removed two crossings and no copies. `Sendfile` removed the copies and halved the calls [S56]. Those are two different savings, and a name that blurs them costs a wrong prediction.

Then the line at the bottom, which is method rather than topic. Name the boundary first: "I flushed and I still lost data" is ambiguous across four of them [V44]. Every number on a slide today came from a run in this room or from a capture in the demo folder [S52] [S53] [S56]. Borrowed numbers are somebody else's measurement, and each one is cited [S14].

Four words to carry out of the room. Whose done did you measure [V50].

117 117. What Lecture 4 answers

On screen:

Part 10 · Close

Same as last week. I name it and do not answer it.

Today's window

Your bytes sat in the page cache for five seconds, in memory no process owns [S52].

In part 9

The kernel moved four mebibytes you never held — the same pages [S56].

The question

What is that memory, and whose is it?

Why it is hard

The kernel reclaims those pages on demand, so free memory is a cache [inference] [V72].

Lecture 4 opens with parts 7 to 9 if we miss them.

[S52] [demo/durability/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md). <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

Last slide, and it carries a question rather than an answer. Same as last week.

Today the bytes sat somewhere for five seconds [S52]. This deck called that place the page cache, and the room took the name on trust.

And in Part 9 the kernel moved four mebibytes out of that same place into a pipe [S56]. The program never held them. `sendfile` works by handing out references to those pages. So the thing now met twice, from two directions, is still a name rather than an object.

So. What is that memory? Whose is it? Asked for the free memory on this machine, does the operating system count the page cache as free or as used?

That question is harder than it sounds, and the reason is one sentence from the reading note. Physical memory is a cache, not a container [V72]. So free RAM is mostly a bookkeeping category rather than a resource going to waste [inference].

Lecture 4 is that question, and memory mapping with it.

And if the clock beat the plan today, Lecture 4 opens on the slide headed Lecture 4 starts here. Waiting and sockets come first. Nothing is lost.

Thank you. Questions now, and afterwards.

118 118. References

On screen:

Part 10 · Close

- [S1] *strace(1)* — Linux man-pages — <https://man7.org/linux/man-pages/man1/strace.1.html>
- [S2] *write(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/write.2.html>
- [S3] *fflush(3)* — Linux man-pages — <https://man7.org/linux/man-pages/man3/fflush.3.html>
- [S4] *close(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/close.2.html>
- [S5] C99 draft, §7.19.3 — <https://busybox.net/~landley/c99-draft.html>
- [S6] Built-in Functions — *open()*, Python documentation — <https://docs.python.org/3/library/functions.html#open>
- [S7] Increase `io.DEFAULT_BUFFER_SIZE` — CPython issue 117151 — <https://github.com/python/cpython/issues/117151>

- **[S9]** *winternl.h* reference — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- **[S10]** NtTrace — Native API tracing for Windows — <https://rogerorr.github.io/NtTrace/>
- **[S11]** End-to-End Arguments in System Design — <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- **[S13]** *fsync* — The Open Group Base Specifications — <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>
- **[S14]** Predict-Observe-Explain — <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S15]** The sad history of the Microsoft POSIX subsystem — <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- **[S17]** *io.IOBase.flush()* — Python documentation — <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- **[S18]** Process Monitor — Sysinternals — <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>
- **[S19]** Inside Native Applications — <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- **[S20]** *os* — Miscellaneous operating system interfaces, *os.write()*, Python documentation — <https://docs.python.org/3/library/os.html#os.write>
- **[S21]** *fread(3)*, the page that documents *fwrite(3)* — Linux man-pages — <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- **[S22]** Windows API index — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- **[S31]** *ltrace(1)* — Linux man-pages — <https://man7.org/linux/man-pages/man1/ltrace.1.html>
- **[S32]** *ltrace* — project page — <https://ltrace.org/>
- **[S33]** NtTrace — source repository — <https://github.com/rogerorr/NtTrace>
- **[S34]** NtTrace64 capture of *cmd.exe /c echo Hello World!* — <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/NtTrace64-CMD-ECHO-Output.txt>
- **[S36]** NtTrace64 capture of *iodemo.exe* — <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/NtTrace64-IODEMO-Output.txt>
- **[S37]** *iodemo* on Linux and on Windows, the recorded runs (*demo/iodemo-c/README.md*) — <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/README.md>

- **[S38]** *CloseHandle function (handleapi.h)* — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/handleapi/nf-handleapi-close-handle>
- **[S39]** *demo/close/README.md* — the double-close capture on both platforms, 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/close/README.md>
- **[S40]** *FlushFileBuffers function (fileapi.h)* — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers>
- **[S41]** *fsyncgate* — the pgsql-hackers thread, March 2018 — <https://danluu.com/fsyncgate/>
- **[S42]** *Can Applications Recover from fsync Failures?* — USENIX ATC 2020 — <https://www.usenix.org/system/files/atc20-rebello.pdf>
- **[S43]** *PRAGMA synchronous* — SQLite documentation — https://www.sqlite.org/pragma.html#pragma_synchronous
- **[S44]** *Delayed allocation and the zero-length file problem* — Theodore Ts'o (2009) — <https://thunk.org/tytso/blog/2009/03/12/delayed-allocation-and-the-zero-length-file-problem/>
- **[S45]** *Delayed allocation and the zero-length file problem* — Valerie Aurora, LWN (2009) — <https://lwn.net/Articles/351422/>
- **[S46]** *epoll(7)* — Linux man-pages — <https://man7.org/linux/man-pages/man7/epoll.7.html>
- **[S47]** *select(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/select.2.html>
- **[S48]** *The C10K problem* — Dan Kegel (current revision) — <http://www.kegel.com/c10k.html>
- **[S49]** *Comparing and Evaluating epoll, select, and poll Event Mechanisms* — Gammo et al. (2004) — <https://www.kernel.org/doc/ols/2004/ols2004v1-pages-215-226.pdf>
- **[S50]** *send(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/send.2.html>
- **[S51]** *Classroom demonstrations: Learning tools or entertainment?* — Crouch, Fagen, Callan and Mazur (2004) — <https://works.swarthmore.edu/fac-physics/203/>
- **[S52]** *demo/durability/README.md* — the writeback timeline and the four-mode counts, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

- **[S53]** *demo/waiting/README.md* — the polling and epoll system-call counts, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>
- **[S54]** *demo/iodemo-c/README.md* — the pipe run and the traced netcat pair, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>
- **[S55]** The C10K problem — Dan Kegel, Internet Archive snapshot of 8 May 1999 — <https://web.archive.org/web/19990508164301/http://www.kegel.com/c10k.html>
- **[S56]** *demo/scatter/README.md* — the writev trace and the sendfile counts, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>
- **[S57]** *demo/destinations-win32/README.md* — the three-destination NtTrace capture, 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/destinations-win32/README.md>
- **[S58]** *writev(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/writev.2.html>
- **[S59]** *pipe(7)* — Linux man-pages — <https://man7.org/linux/man-pages/man7/pipe.7.html>
- **[S60]** *sendfile(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/sendfile.2.html>
- **[S61]** Module *ngx_http_core_module* — nginx documentation — https://nginx.org/en/docs/http/ngx_http_core_module.html#sendfile
- **[S62]** *WriteFileGather* function (*fileapi.h*) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefilegather>
- **[S63]** *TransmitFile* function (*mswsock.h*) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/mswsock/nf-mswsock-transmitfile>
- **[S64]** *socket* — Low-level networking interface, Python documentation — <https://docs.python.org/3/library/socket.html#socket.socket.sendfile>
- **[S65]** *valkey/src/networking.c* — the reply path's writev — <https://github.com/valkey-io/valkey/blob/00a8a19554a39facb591837ea41ba4350260919d/src/networking.c>
- **[S66]** *strace Wow Much Syscall* — Gregg (2014). <https://www.brendangregg.com/blog/2014-05-11/strace-wow-much-syscall.html>
- **[S67]** *fsync(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/fsync.2.html>
- **[S68]** Linux kernel documentation — *admin-guide/sysctl/vm.rst* — <https://www.kernel.org/doc/html/latest/admin-guide/sysctl/vm.html>

- **[S69]** WSASend function (winsock2.h) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/winsock2/nf-winsock2-wsasend>
- **[S70]** rename(2) — Linux man-pages — <https://man7.org/linux/man-pages/man2/rename.2.html>
- **[S71]** PostgreSQL commit 9ccdd7f — "PANIC on fsync() failure" (2018) — <https://git.postgresql.org/gitweb/?p=postgresql.git;a=commit;h=9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1>
- **[S72]** Named Pipes — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/ipc/named-pipes>
- **[S73]** PeekNamedPipe function (namedpipeapi.h) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/namedpipeapi/nf-namedpipeapi-peeknamedpipe>
- **[S74]** GetQueuedCompletionStatus function (ioapiset.h) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/ioapiset/nf-ioapiset-getqueuedcompletionstatus>
- **[S75]** io_uring(7) — Linux man-pages — https://man7.org/linux/man-pages/man7/io_uring.7.html
- **[V4]** A Successful write() Is a Handoff, Not a Save — The idiom the room already writes contains no durability step.
- **[V5]** A Successful write() Is a Handoff, Not a Save — The limit on the thesis. The bottom of the stack is not observable from software, and the deck says so.
- **[V6]** Standard output is buffered differently depending on where it points — Removes the misconception that a Python write passes through the C library buffer.
- **[V8]** Windows and Linux place the stable syscall boundary at different layers — Takeaway 4, stated as a placement question rather than a quality judgement.
- **[V9]** Windows and Linux place the stable syscall boundary at different layers — Stops section 5 reading as a complaint about Windows.
- **[V10]** The System Call Boundary — Makes "a library call is not a system call" falsifiable with two tools instead of asserting it.
- **[V11]** The System Call Boundary — The closing honest-limits slide.
- **[V14]** Where the Abstraction Leaks — Bounds section 3's vanishing-output demonstration so it does not teach the misconception it exists to remove.
- **[V17]** Observing a Running System — Why sections 1 and 2 need two runs.
- **[V20]** File IO and Durability — The device reports success before the bytes reach the medium, which is the sentence Lecture 2 stopped on.
- **[V21]** NtTrace — Native API tracing for Windows (Roger Orr) — Why this tool is usable on a student's own machine, and what its design bought.

- **[V22]** *NtTrace — Native API tracing for Windows (Roger Orr) — The single string in the Windows trace that shows the layer difference.*
- **[V23]** *NtTrace — Native API tracing for Windows (Roger Orr) — Comparison slide names the defect it has just cleared.*
- **[V24]** *PostgreSQL's fsync() surprise — Shipped storage engines answer the deferred-error problem by refusing to retry.*
- **[V25]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot — The honest-limits slide. The tool taught here is a teaching and debugging instrument, not an operational one.*
- **[V28]** *Sockets and Network I-O — One-slide bridge to networking.*
- **[V29]** *Sockets and Network I-O — The pipe row on the same slide.*
- **[V30]** *File IO and Durability — The row of the note's per-layer instrumentation table covering the kernel-to-device boundary.*
- **[V33]** *File IO and Durability — The bottom arrow of the stack diagram, and the "emptied by" cell of level 4.*
- **[V36]** *Windows and Linux place the stable syscall boundary at different layers — The caveat on the outlier list.*
- **[V37]** *File IO and Durability — The Windows half of level 3's "emptied by" cell, and of the third arrow in the stack diagram.*
- **[V38]** *Windows and Linux place the stable syscall boundary at different layers — Why Linux nominates the layer it does, and why the other platforms need not.*
- **[V39]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot — What the lower-overhead mode named in [V25] actually buys, on a named workload.*
- **[V40]** *PostgreSQL's fsync() surprise — How many storage engines abandoned the retry, and when.*
- **[V41]** *PostgreSQL's fsync() surprise — The independence of the three engines, which is what makes the never-retry slide an argument rather than an anecdote.*
- **[V43]** *Windows and Linux place the stable syscall boundary at different layers — The named list behind the outlier correction.*
- **[V44]** *File IO and Durability — Core claim, stated as a conjunction.*
- **[V45]** *File IO and Durability — The objection the room raises during section 3 — but I can read the file back.*
- **[V46]** *File IO and Durability — Windows half at equal depth.*
- **[V47]** *File IO and Durability — The honest Windows answer is undocumented, which is a stronger and truer statement than the same or safe.*

- **[V48]** *File IO and Durability — Honesty slide. Why the 40 MiB file was written back sooner than the 4 MiB one, which is the first question the observation provokes.*
- **[V49]** *File IO and Durability — Close. A shipped database declines to trust the last software handoff and defends itself another way.*
- **[V50]** *Where the Abstraction Leaks — The thesis in one sentence, from the vault's own synthesis note.*
- **[V51]** *Where the Abstraction Leaks — Fixable half. The room needs the part software can get right, or the durability section reads as fatalism.*
- **[V52]** *Where the Abstraction Leaks — Counter-position. Trusting the abstraction is respectable, and the note says exactly where it stops being respectable.*
- **[V53]** *A failed fsync() is not retryable — The kernel's done observed lying, in a trace of exactly the kind this course teaches the room to read.*
- **[V54]** *A failed fsync() is not retryable — Why the obvious repair is not a repair. It has the same shape as Lecture 2's slide "Never retry a failed close", one level down.*
- **[V55]** *A failed fsync() is not retryable — Windows slide. The asymmetry is about documentation, not about which platform is safer.*
- **[V56]** *PostgreSQL's fsync() surprise — The industry answer to an untrustworthy done is crash and replay, and it was never meant to last this long.*
- **[V57]** *Readiness and Completion Models — Buffer ownership, not API taste, is what separates the two models, and it is why porting between them is a rewrite.*
- **[V58]** *Readiness and Completion Models — The sentence that stops the room reading ready as a guarantee, which is the misconception this model produces.*
- **[V59]** *Readiness and Completion Models — Correction to its own framing.*
- **[V60]** *Readiness and Completion Models — Windows slide. What the room should carry away as present tense, and what is only direction of travel.*
- **[V61]** *The C10K Problem Was About the Cost of Waiting, Not I-O — Predict slide. The controlled experiment behind the question, stated as what was held fixed and what was varied.*
- **[V62]** *The C10K Problem Was About the Cost of Waiting, Not I-O — Explanation. `epoll` did not make a read faster, and the room needs that said before it draws the wrong conclusion from the counts.*
- **[V63]** *The C10K Problem Was About the Cost of Waiting, Not I-O — The page most often cited for this argument did not make it in 1999, so the deck cites the revision and the measurement instead.*
- **[V64]** *Sockets and Network I-O — Mechanism. Why read and write work on a socket, stated as a kernel structure rather than as a convention.*

- **[V65]** *Sockets and Network I-O — Boundary. The same interface does not mean the same contract, and the note names where the default stops.*
- **[V66]** *Sockets and Network I-O — Windows slide. The platforms agree on the mechanism and disagree on which layer they promise, which is the accurate statement.*
- **[V67]** *Interprocess Communication — Boundary. The clean one-write-one-read result in the capture holds because fourteen bytes is small, and the room should know the threshold.*
- **[V68]** *io_uring and Windows IoRing — Completion on Both Platforms — Close. Where both platforms ended up, stated precisely enough not to read as "they converged".*
- **[V69]** *Blocking Is a Scheduler Decision, Not a Property of the I-O — The one question that orders polling, readiness and completion into a ladder rather than a feature list.*
- **[V70]** *Network Buffering and Delivery Semantics — Boundary. The capture shows one write and one read only because the payload is fourteen bytes, and the slide must say so.*
- **[V71]** *Standard output is buffered differently depending on where it points — Honest bound on its own demonstration.*
- **[V72]** *Virtual Memory and the Page Cache — Handover. The page cache held the bytes for five seconds, and what that memory actually is belongs to the next lecture.*
- **[V73]** *io_uring and Windows IoRing — Completion on Both Platforms — Boundary. The demonstration uses pipes, and a student who tries the same loop on a file gets a mechanism that always answers yes.*
- **[V74]** *Readiness and Completion Models — What a completion model demands of the program, in Microsoft's own words, and the memory bill that follows from it.*
- **[V75]** *Readiness and Completion Models — The other half of the ownership comparison.*
- **[V76]** *Readiness and Completion Models — Why the choice of model is structural rather than a matter of API taste.*
- **[V77]** *Readiness and Completion Models — Takeaway 2 and section 4's definition slide.*
- **[V78]** *File IO and Durability — What a dirty page is, what writeback is, and that the kernel rather than the program schedules it.*
- **[V79]** *io_uring and Windows IoRing — Completion on Both Platforms — Close. The version in which Linux acquired a completion interface.*

- **[V80]** *io_uring and Windows IoRing — Completion on Both Platforms — Close. The build in which Windows acquired a ring, paired with [V79].*
- **[V81]** *Readiness and Completion Models — Correction. The control case that shows readiness is the Unix family's answer rather than a Linux quirk.*
- **[V82]** *Interprocess Communication — Boundary. The threshold in bytes, and what happens above it, which the atomicity span [V67] does not state.*
- **[V83]** *A failed fsync() is not retryable — Windows slide. The mechanism behind the undocumented contract, which is why no single answer exists.*
- **[V84]** *PostgreSQL's fsync() surprise — What InnoDB actually changed, which the count-and-year span [V40] does not carry.*
- **[V85]** *A failed fsync() is not retryable — Close. The log-and-replay answer, in PostgreSQL's own shipped documentation rather than as the deck's summary of it.*
- **[V86]** *File IO and Durability — The device lying is a reported practice rather than a theoretical possibility, and a shipped database says so.*
- **[V87]** *Sockets and Network I-O — Boundary. A concrete operation that does not transfer, which the strong-default span [V65] asserts without instancing.*
- **[V88]** *io_uring and Windows IoRing — Completion on Both Platforms — Why the regular-file gap mattered enough to build a second model, which the epoll -and-regular-files span [V73] does not say.*
- **[V89]** *A failed fsync() is not retryable — The mechanism behind the failed retry, which [V54]'s one-sentence span states without explaining.*
- **[V90]** *A failed fsync() is not retryable — How long the misreading stood before anyone noticed, which is what makes it a contract failure rather than a careless bug.*
- **[V91]** *File IO and Durability — Windows slide. The judgement itself, from the note that makes it, rather than asserted by the deck on the strength of a missing wiki row.*
- **[V92]** *Readiness and Completion Models — The select(2) BUGS caveat applies to the interface the demonstration actually used.*
- **[V93]** *PostgreSQL's fsync() surprise — What PostgreSQL actually changed, and that the change reached every release rather than only the current one.*
- **[V94]** *The C10K Problem Was About the Cost of Waiting, Not I-O — Kegel's arithmetic for why a thread per connection ran out of virtual memory.*
- **[V95]** *The C10K Problem Was About the Cost of Waiting, Not I-O — Banga, Mogul and Druschel on the scheduler half of the same bill.*
- **[V96]** *The C10K Problem Was About the Cost of Waiting, Not I-O — What unifies the thread rung with the polling rung: a charge levied per connection, not per byte.*

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

Every source behind every claim today, in the order the deck used them.

The S entries are external, and every one carries a link to follow tonight. Most of them are manual pages, vendor documentation and standards. They are short, they are free, and the people who define the behaviour wrote them.

Several of the S entries are this course's own captures: the Windows traces and the container runs. Those ship with the deck, in the demo folder, so the numbers can be checked against the files rather than against the slides.

The V entries are notes from the author's private research vault. Those are not open to the room, so the handout carries a one-page summary of each one together with every external source it draws on.